

RISC-V rv32i CPU 설계 및 simulation

이준형



목차

1. RISC – V 개요
2. B/D
3. Type별 동작 방식
 - a. R -type
 - b. I -type
 - c. S -type
 - d. IL -type
 - e. B -type
 - f. U -type
 - g. J -type
4. C to Assembler 분석
5. Trouble Shooting



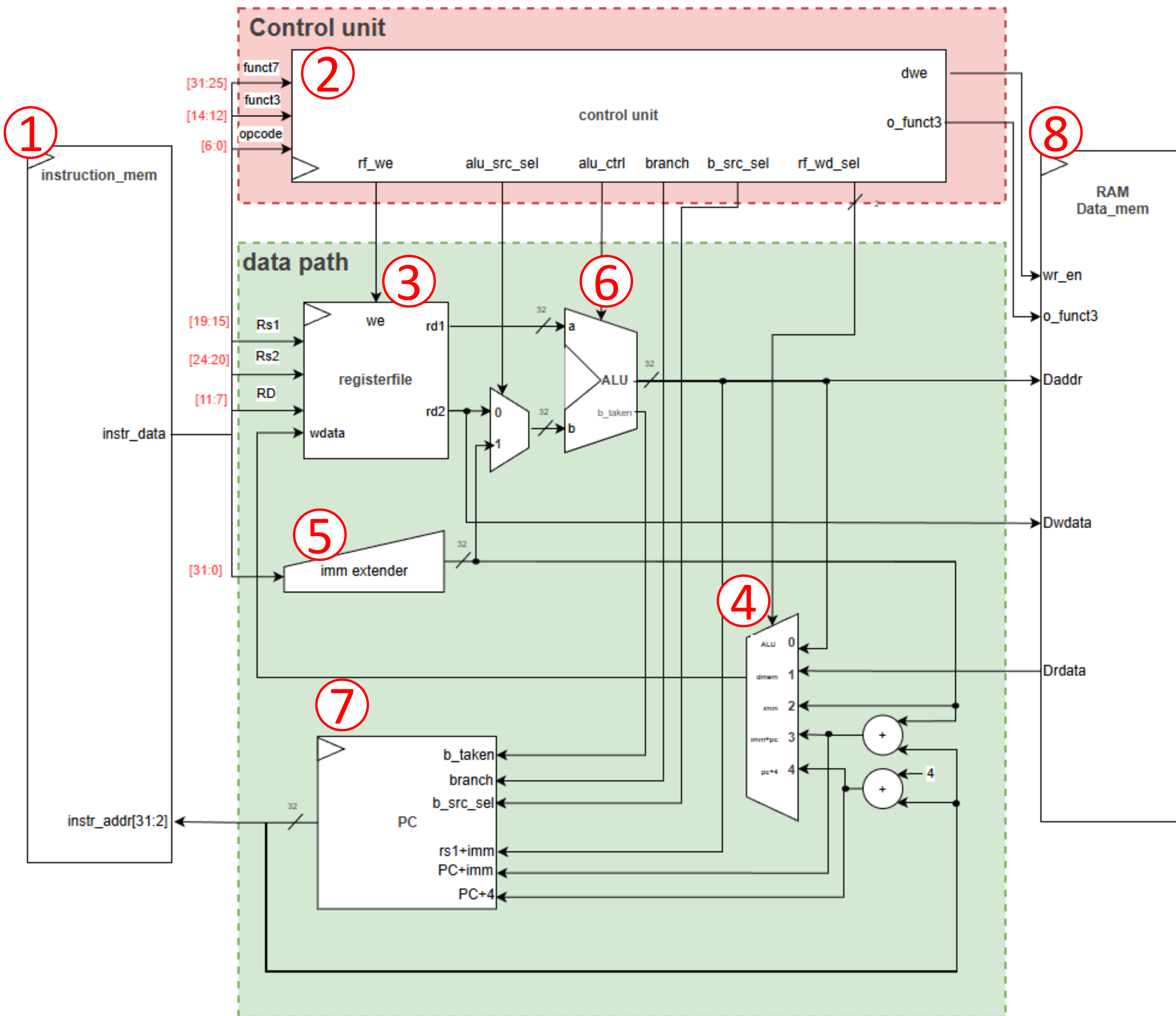
1. RISC – V 개요

RISC-V 란:

오픈 소스 명령어 집합 아키텍처(ISA)로,
최소한의 필수 명령어만 유지하는 것이 특징

RISC- V rv32i		
명령어 set	32bit 고정 길이	
Register file	32개 구성	
아키텍처		
방식	Von Neuman아키텍처	Harvard 아키텍처
특징	Data와 명령어가 하나의 메모리에 저장	Data와 명령어 메모리가 분리되어 저장

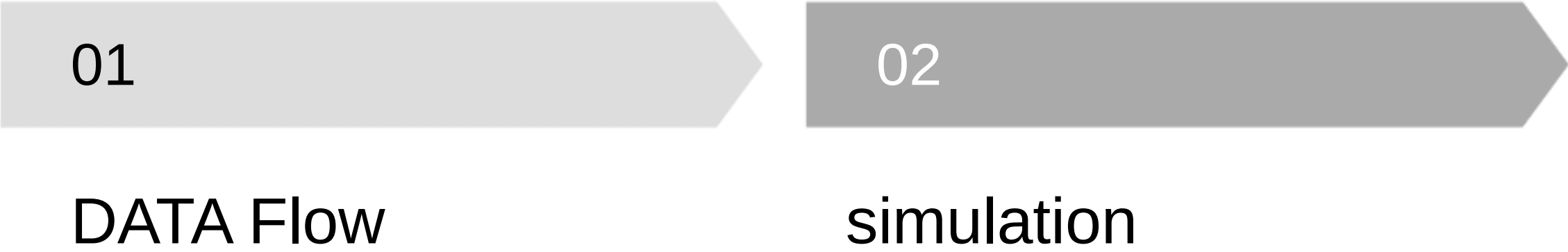
2. B/D



	Module	part	기능
1	Instruction Mem	ROM	실행 명령어 메모리
2	Control Unit	Control Unit	전체적인 System 제어
3	Register File	Data path	ALU 연산을 위한 register
4	5-input MUX		Register File Write Data MUX
5	Imm Extender		32 bit 상수로 변환
6	ALU		연산처리 & 분기 확인
7	PC	RAM	Instruction Mem 다음 주소 지정
8	DATA memory		Load & STORE data 메모리 (word addressing)

3. Type 별 동작 방식

설명순서	
1	R – type
2	I - type
3	S - type
4	IL - type
5	B - type
6	U - type
7	J - type



3. R-type

	ADD	SUB	SLL	SLT	SLTU	XOR	SRL	SRA	OR	AND
Funct7 [5]	0	1	0					1	0	
funct3	0		1	2	3	4	5		6	7

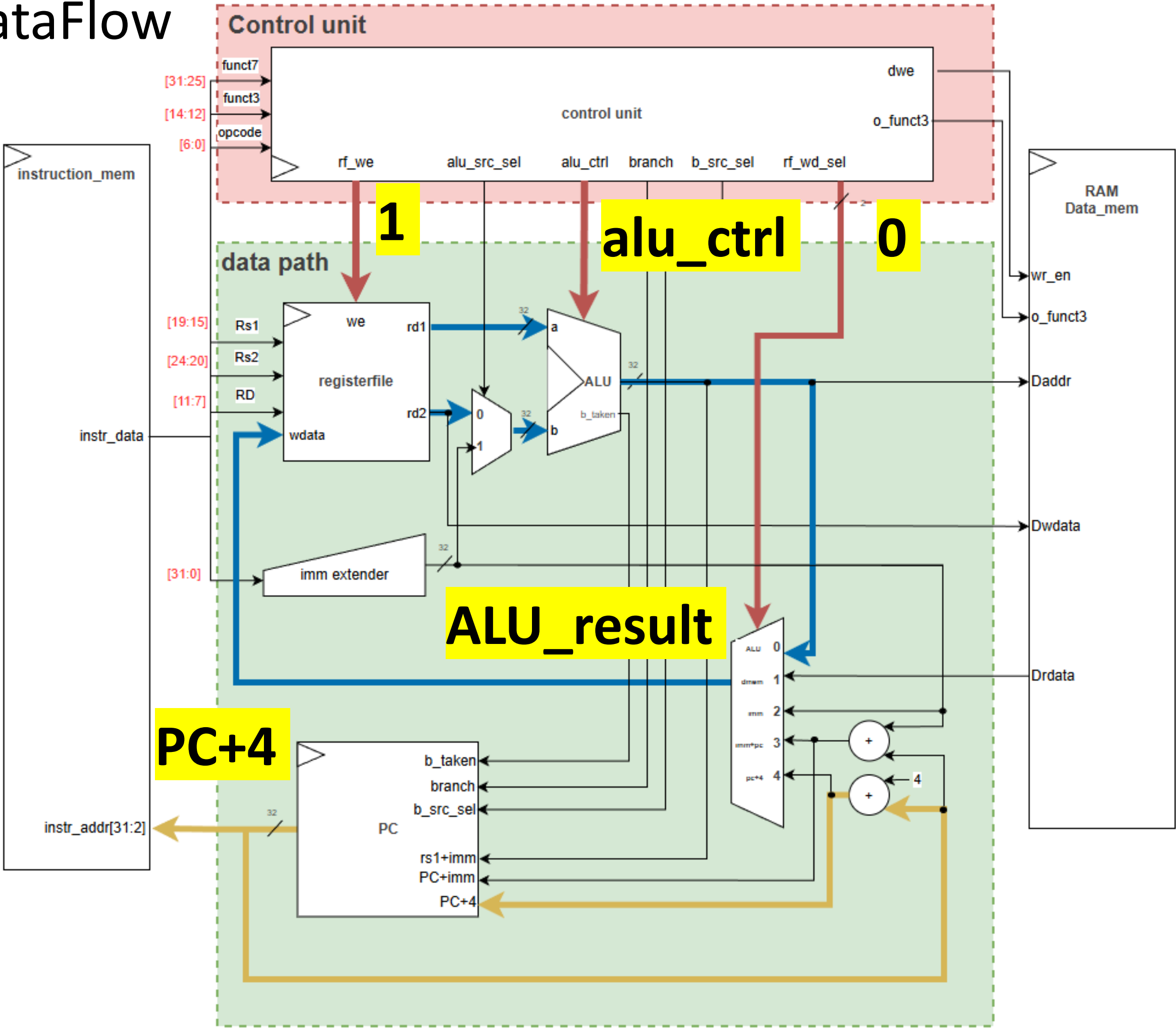
- Rs1, Rs2을 산술 연산하는 명령어
- 연산자는 Funct3, Funct7에 의해 결정

R-type

Funct7	Rs2	Rs1	Funct3	Rd	OPCODE(R-type)
--------	-----	-----	--------	----	----------------

3. R-type

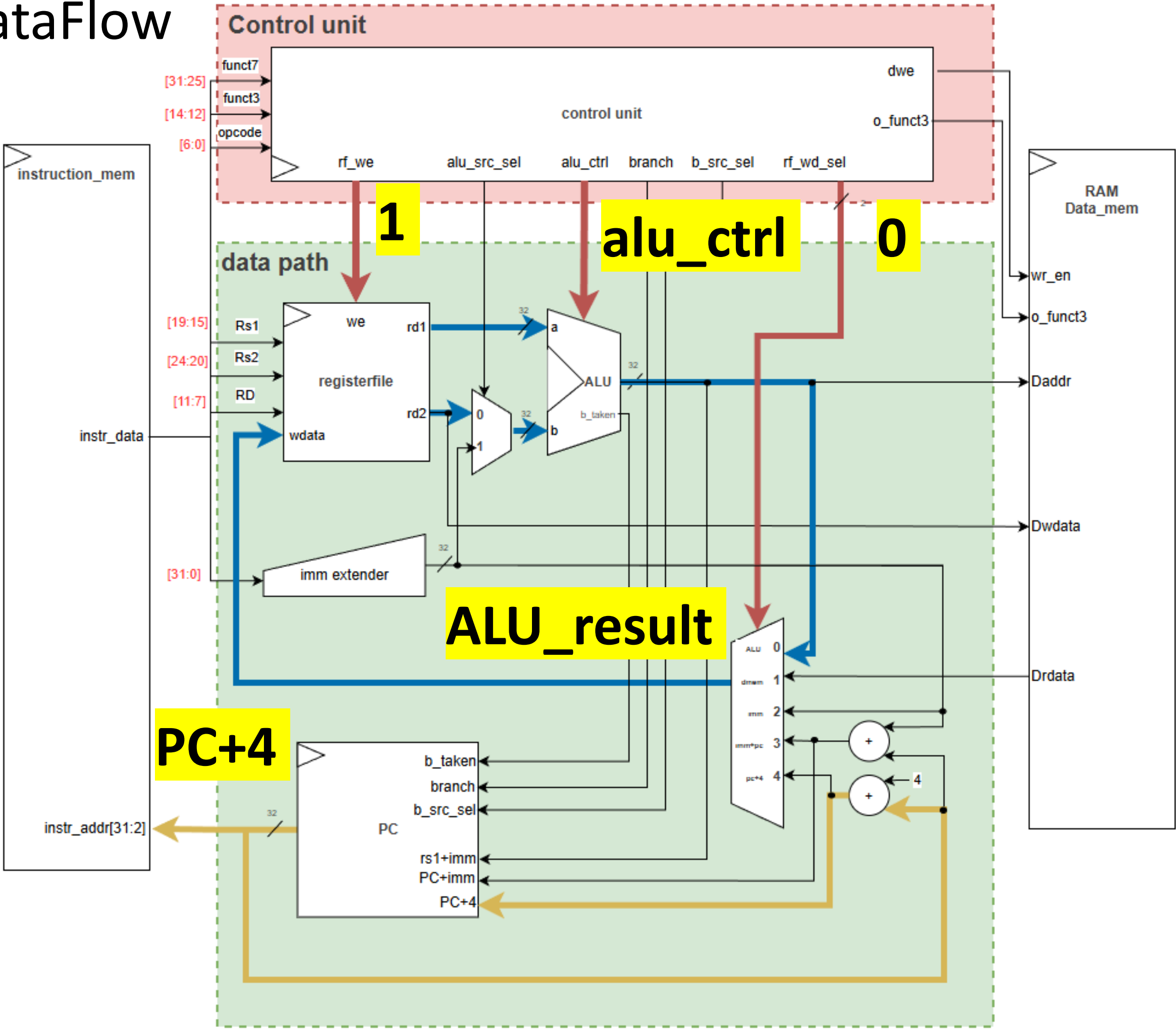
• DataFlow



control_unit	
OUTPUT	R_type
rf_we	1
alu_src_sel	0
rf_wd_sel	0
alu_control [b_taken,ALU_ctrl]	0,funct7[5],funct3
b_src_sel	0
branch	0
o_funct3	0
dwe	0

3. R-type

• DataFlow



ALU											
ALU_control		ADD	SUB	SLL	SLT	SLTU	XOR	SRL	SRA	OR	AND
[4]	branch	0									
[3]	Funct7 [5]	0	1	0				1	0		
[2:0]	funct3	0		1	2	3	4	5		6	7
output											
Result		a + b	a - b	a<<b	(a<b) ?1:0	(a<b)	a ^ b	a>>b	a>>> b	a b	a & b
Signed					S				S		
B_taken		0									

R-type

Funct7

Rs2

Rs1

Funct3

Rd

OPCODE(R-type)

3. R-type

- simulation (시나리오)

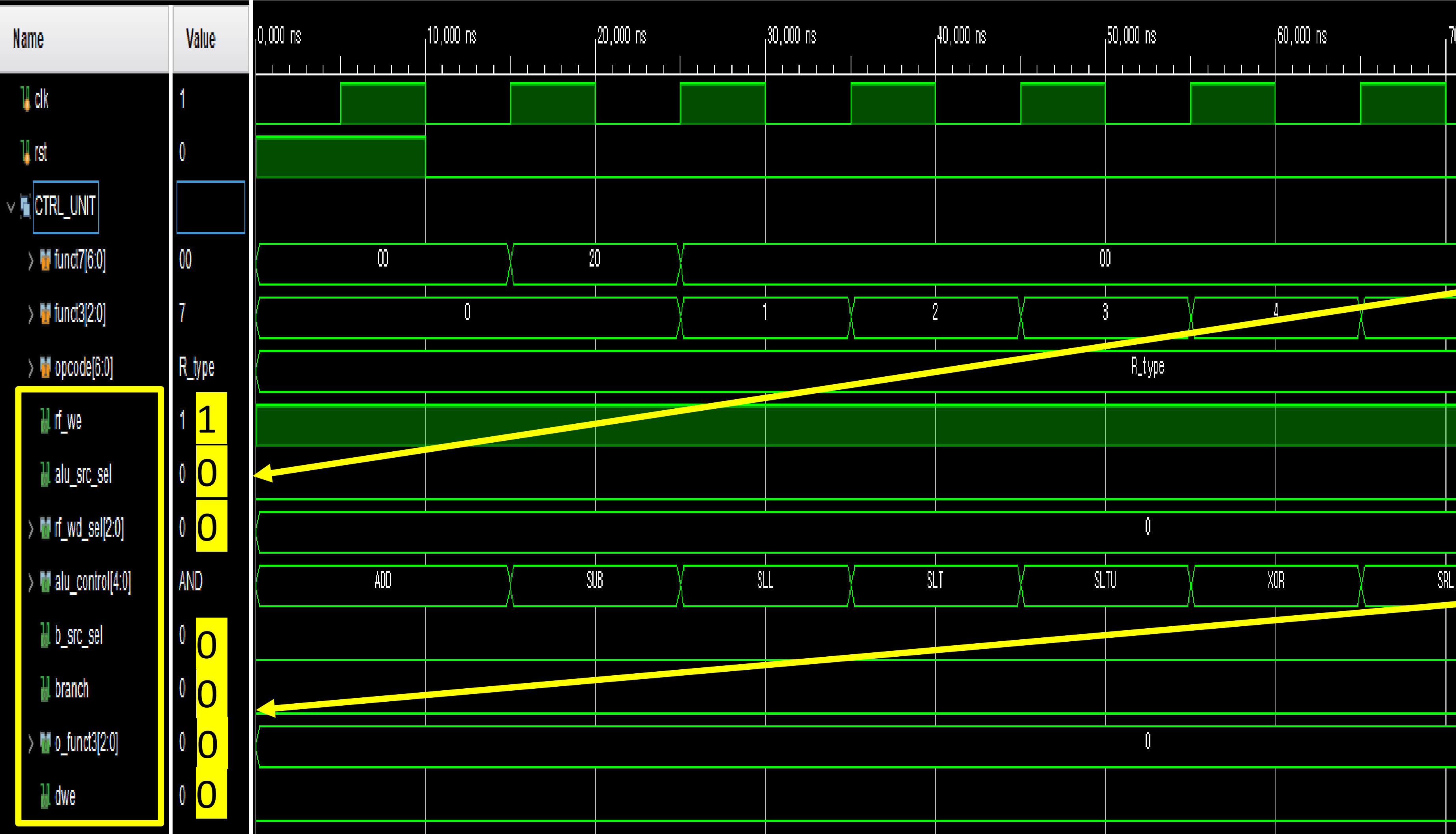
Register File			
addr	name	value	용도
1	rs1	-100	Full case
2	rs2	200	Shfit 연산을 제외한 나머지 case
3	Shift	2	SLL, SRL,SRA
4	rd	Alu_result	

INSTR MEM		
PC	INSRT	Result(DEX)
0	ADD	100
1	SUB	-300
2	SLL	-400
3	SLT	1
4	SLTU	0
5	XOR	-172
6	SRL	1073741799
7	SRA	-25
8	OR	-36
9	AND	136

R-type	Funct7	Rs2	Rs1	Funct3	Rd	OPCODE(R-type)
--------	--------	-----	-----	--------	----	----------------

3. R-type

- simulation(Control Unit)

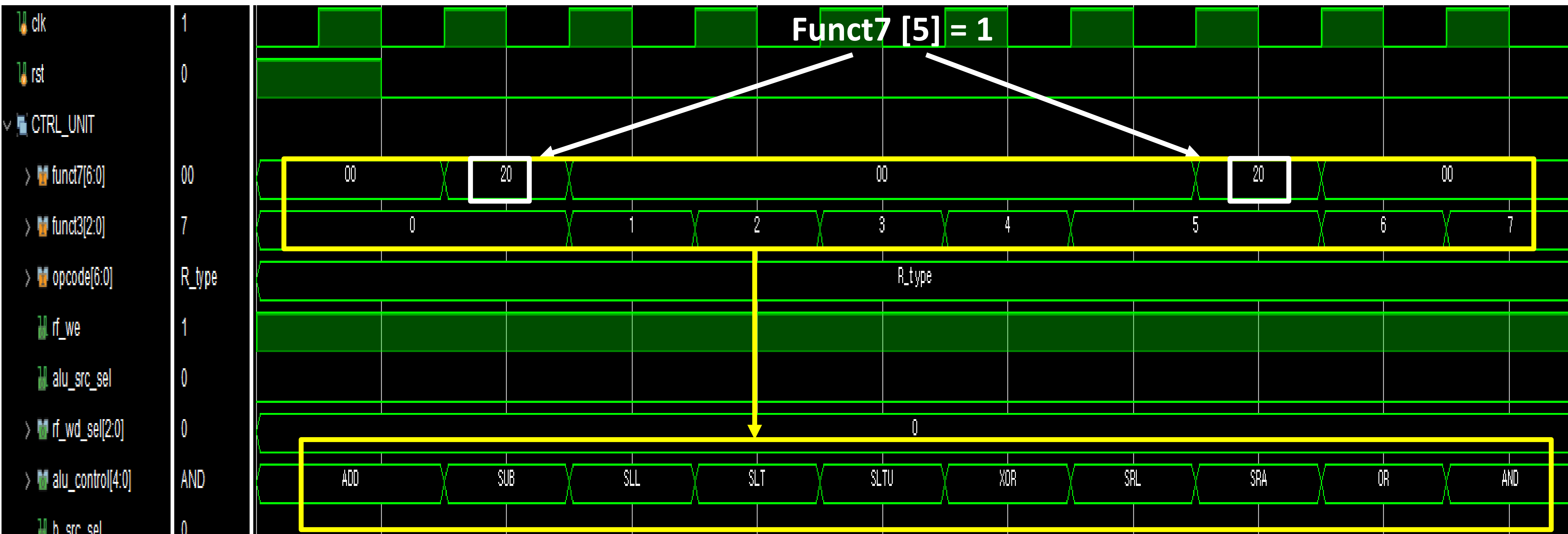


control_unit	
OUTPUT	R_type
rf_we	1
alu_src_sel	0
rf_wd_sel	0
alu_control	0,func7[5],func3
[b_taken,ALU_ctrl]	
b_src_sel	0
branch	0
o_func3	0
dwe	0

3. R-type

- simulation(Control Unit)

	ADD	SUB	SLL	SLT	SLTU	XOR	SRL	SRA	OR	AND
funct7[5]	0	1	0					1	0	
funct3	0		1	2	3	4	5		6	7



3. R-type

- simulation(ALU) - ADD ~ SLTU

INSTR MEM		
PC	INSRT	Result(DEX)
0	ADD	100
1	SUB	-300
2	SLL	-400
3	SLT	1
4	SLTU	0
5	XOR	-172
6	SRL	1073741799
7	SRA	-25
8	OR	-36
9	AND	136

ALU

> a[31:0]

-100

> b[31:0]

200

> alu_control[4:0]

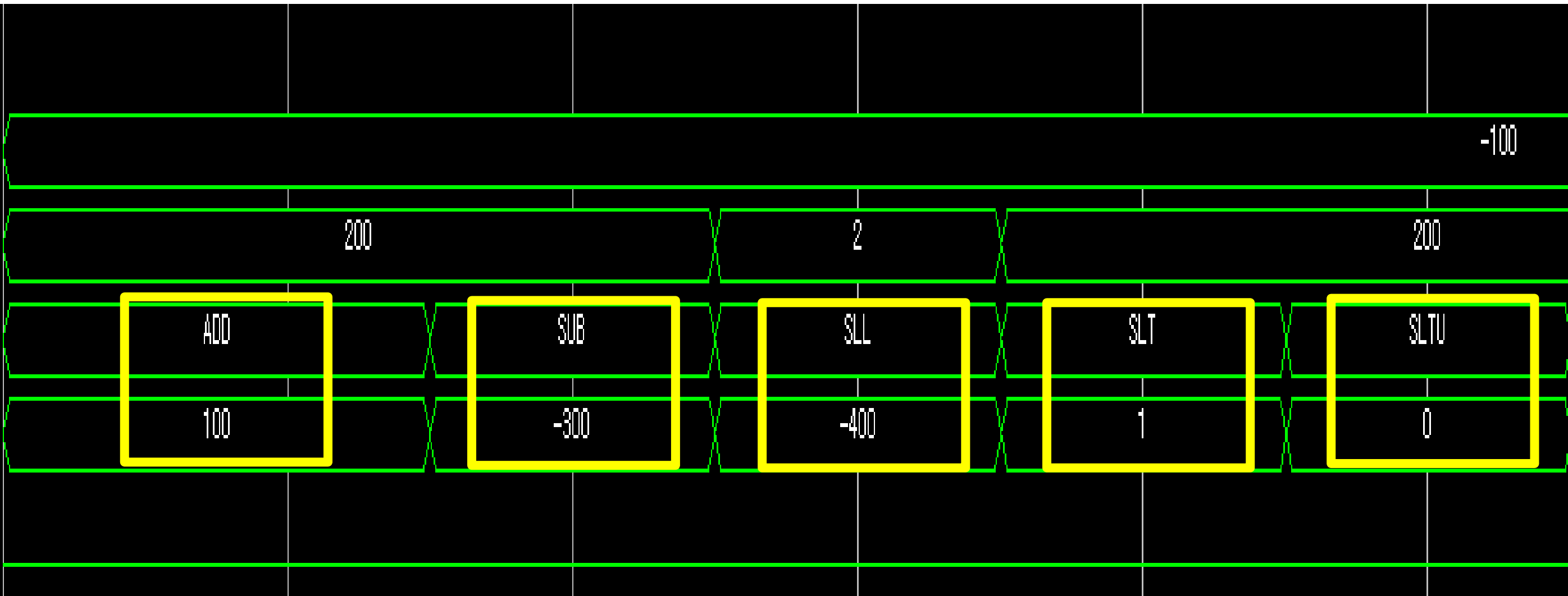
XOR

> alu_result[31:0]

-172

b_taken

0



- XOR ~ AND

ALU

> a[31:0]

-100

> b[31:0]

200

> alu_control[4:0]

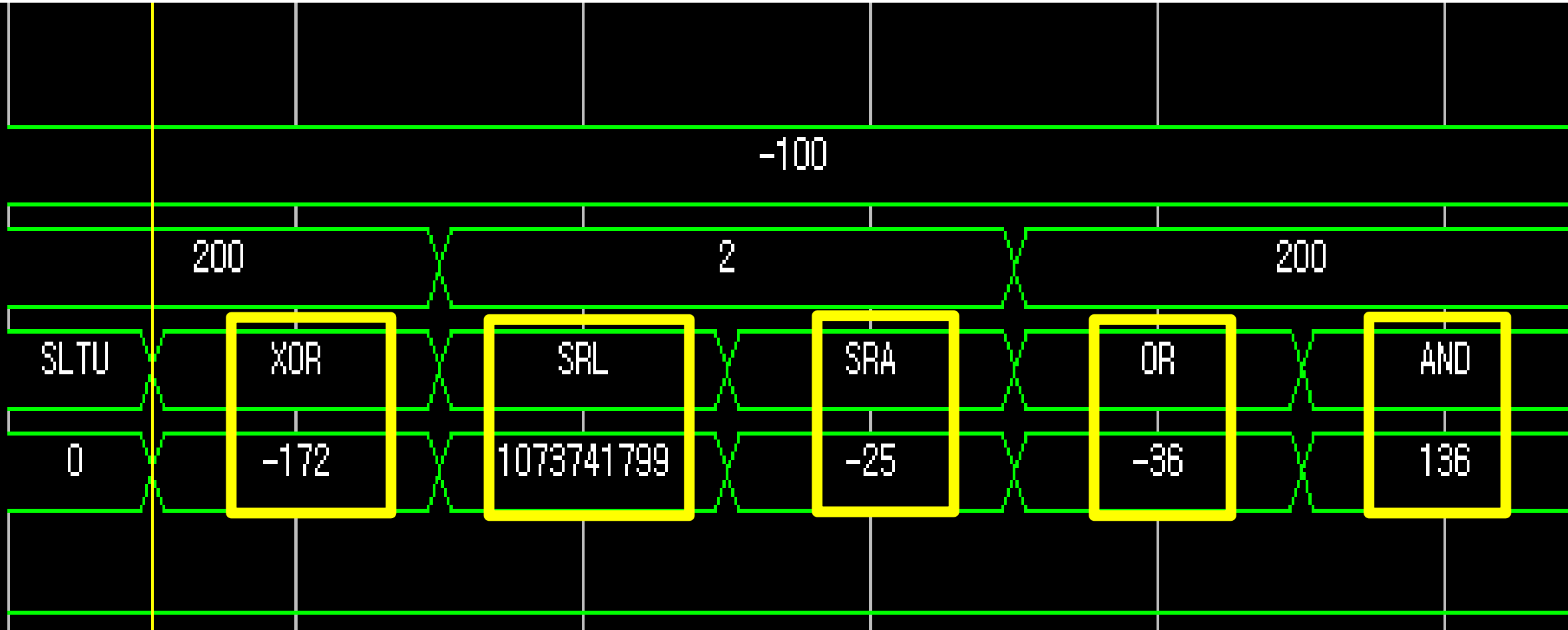
XOR

> alu_result[31:0]

-172

b_taken

0



3. R-type

- simulation(INSTR mem & PC)



3. I-type

	ADDI	SLLI	SLTI	SLTUI	XORI	SRLI	SRAI	ORI	ANDI
funct7[5]	x	0	x			0	1	x	
funct3	0	1	2	3	4	5		6	7

- Rs1, 상수 imm을 산술 연산하는 명령어
- shift 연산자 이외의 명령어는 Funct3 의해 결정
- Shift 연산자는 Funct7, Funct3에 의해 결정

I-type

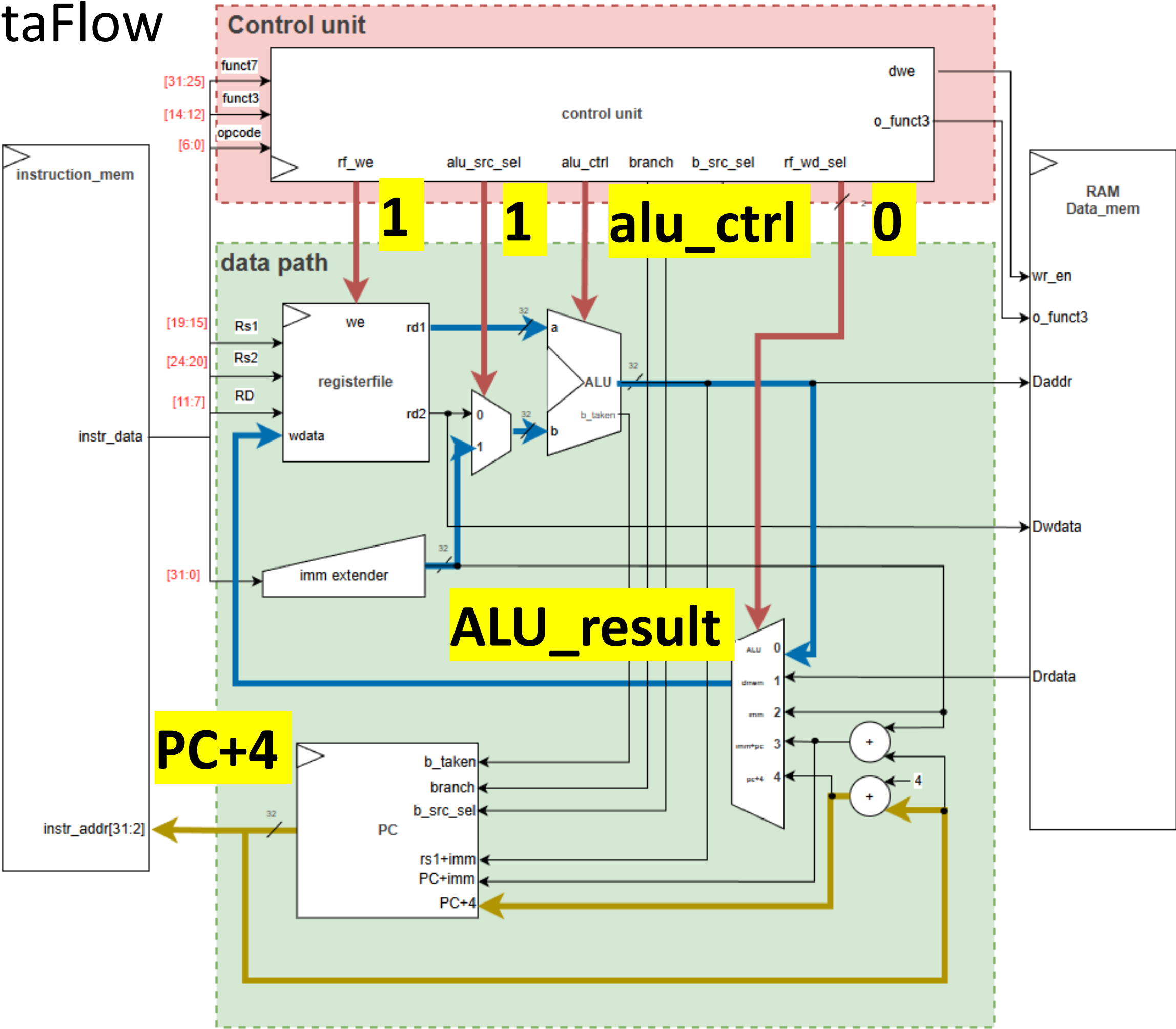
Imm[11:0]	Rs1	Funct3	Rd	OPCODE(I-type)
-----------	-----	--------	----	----------------

I - shift 명령어

funct7	shamt	Rs1	Funct3	Rd	OPCODE(I-type)
--------	-------	-----	--------	----	----------------

3. I-type

• DataFlow



control_unit		
OUTPUT		I_type
rf_we		1
alu_src_sel		1
rf_wd_sel		0
alu_control	SLL, SRL, SRA	0, funct7[5], funct3
[b_taken, ALU_ctrl]	other	0, 0, funct3
b_src_sel		0
branch		0
o_funct3		0
dwe		0

shift 명령어:
funct7[5]의해 동작
other 명령어:
funct7[5] 관계없음

I-type

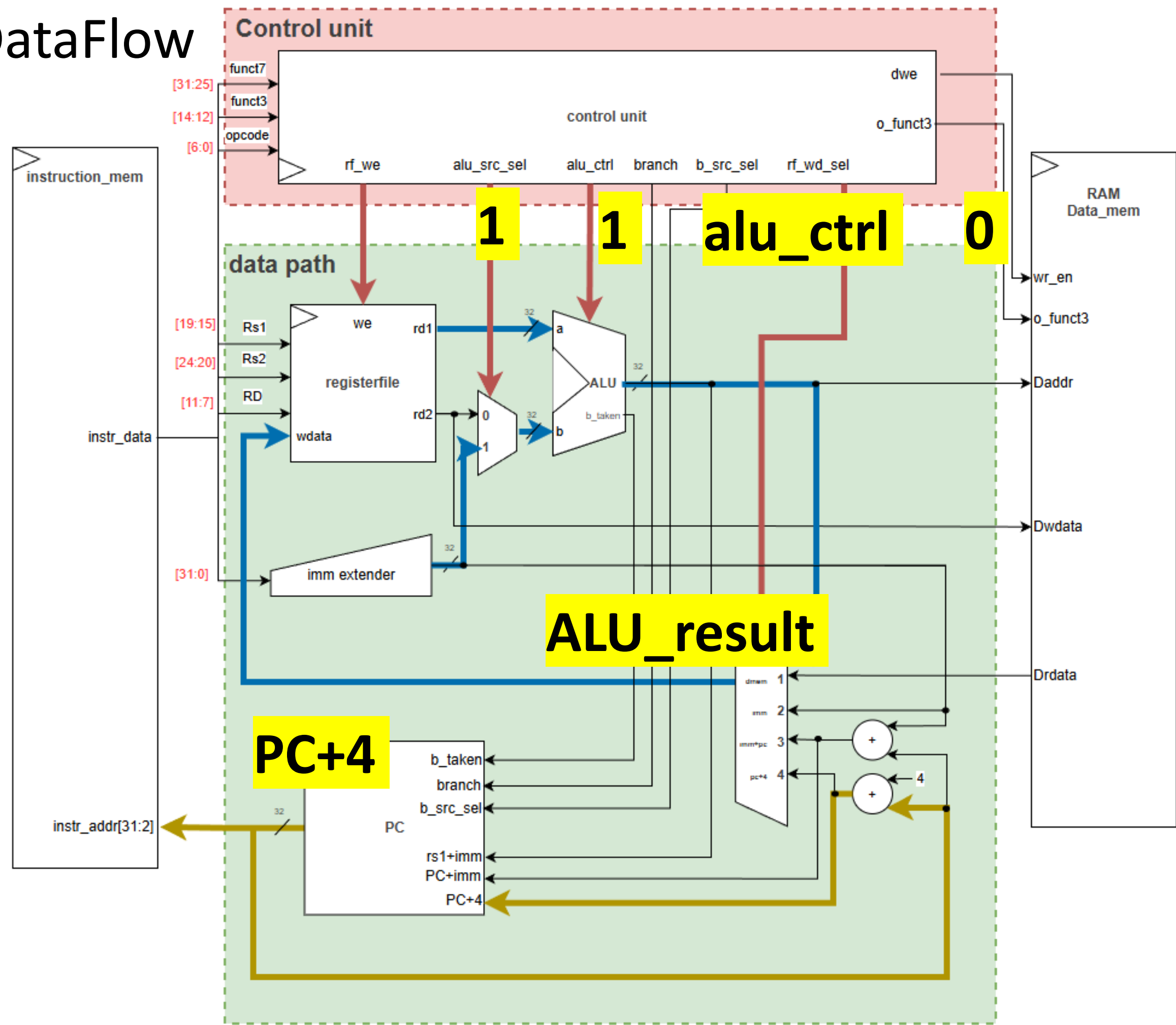
Imm[11:0]	Rs1	Funct3	Rd	OPCODE(I-type)
-----------	-----	--------	----	----------------

I-shift 명령어

funct7	shamt	Rs1	Funct3	Rd	OPCODE(I-type)
--------	-------	-----	--------	----	----------------

3. I-type

• DataFlow



ALU										
ALU_control		ADD	SLL	SLT	SLTU	XOR	SRL	SRA	OR	AND
[4]	branch	0								
[3]	Funct7 [5]	0						1	0	
[2:0]	funct3	0	1	2	3	4	5		6	7
output										
Result		$a + b$	$a \ll b$	$(a < b) ? 1 : 0$	$(a < b)$	$a \wedge b$	$a \gg b$	$a \ggg b$	$a b$	$a \& b$
Signed				S				S		
B_taken		0								

I-type

Imm[11:0]

Rs1

Funct3

Rd

OPCODE(I-type)

I-shift 명령어

funct7

shamt

Rs1

Funct3

Rd

OPCODE(I-type)

3. I-type

- simulation(시나리오)

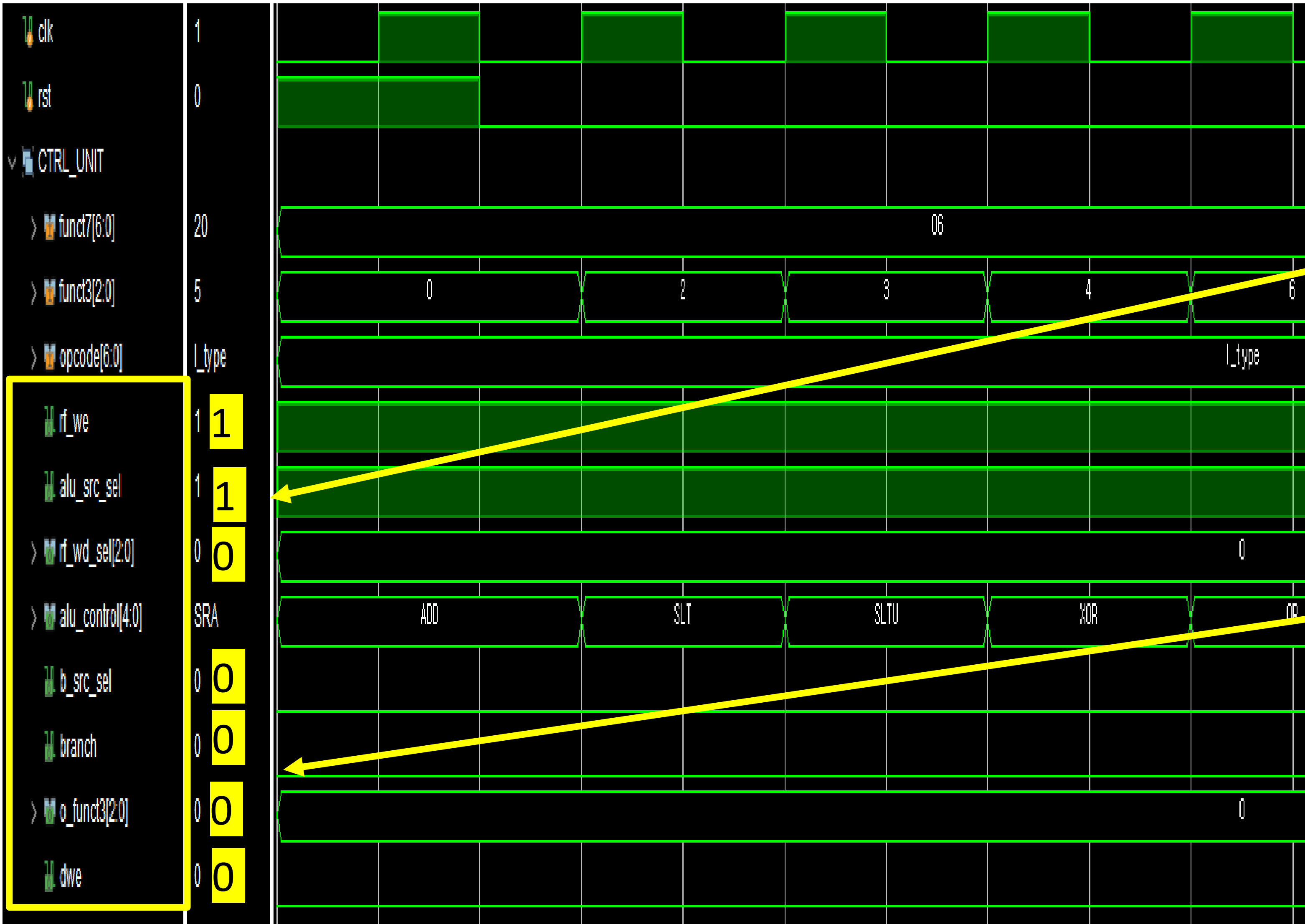
Register File			
addr	name	value	용도
1	rs1	-100	Full case
2	rs2	200	Shfit 연산을 제외한 나머지 case
3	Shfit	2	SLTI,SLTUI,SRLI,SRAI
4	rd	Alu_result	

INSTR MEM		
PC	INSRT	Result(DEX)
0	ADDI	100
1	SLTI	1
2	SLTUI	0
3	XORI	-172
4	ORI	-36
5	ANDI	136
6	SRLI	1073741799
7	SRAI	-25

I-type	Imm[11:0]		Rs1	Funct3	Rd	OPCODE(I-type)
I-shift 명령어	funct7	shamt	Rs1	Funct3	Rd	OPCODE(I-type)

3. I-type

- simulation(Control Unit)

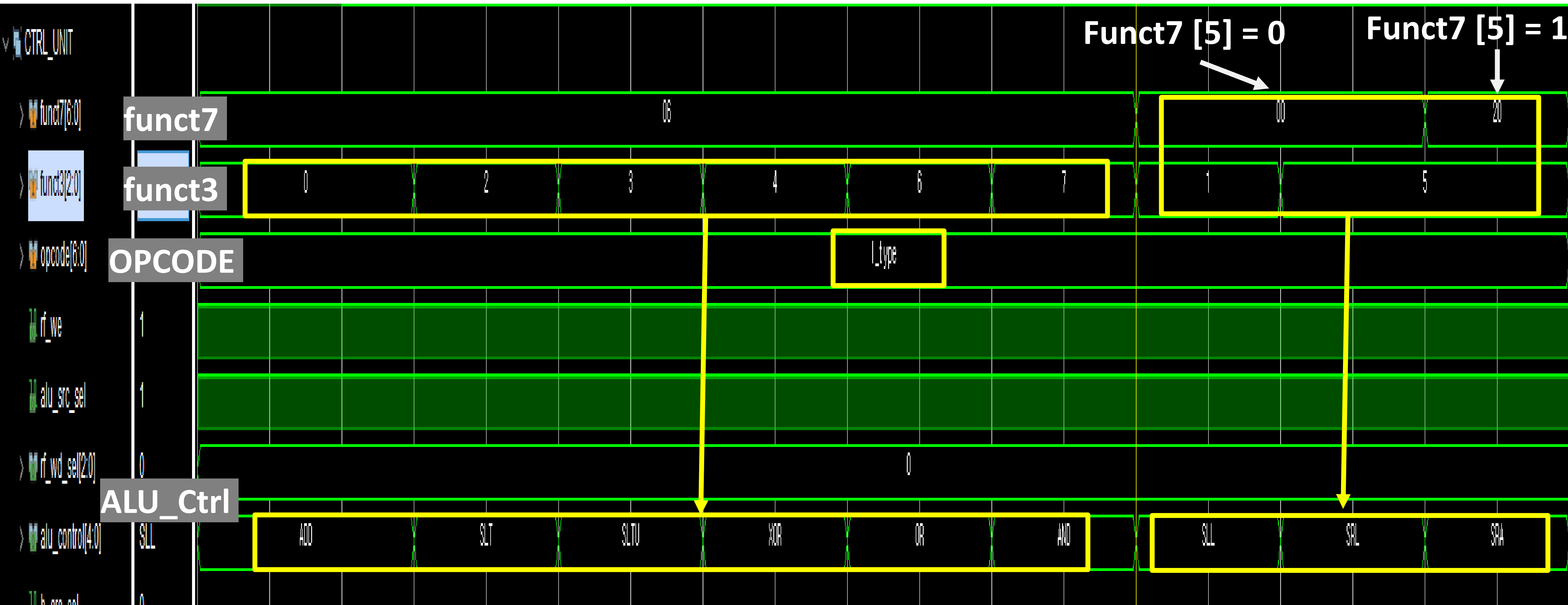


control_unit		
OUTPUT		I_type
rf_we		1
alu_src_sel		1
rf_wd_sel		0
alu_control	SLL SRL,SRA	0,funct7[5],funct3
	[b_taken,ALU_ctrl] other	0,0,funct3
b_src_sel		0
branch		0
o_funct3		0
dwe		0

3. I-type

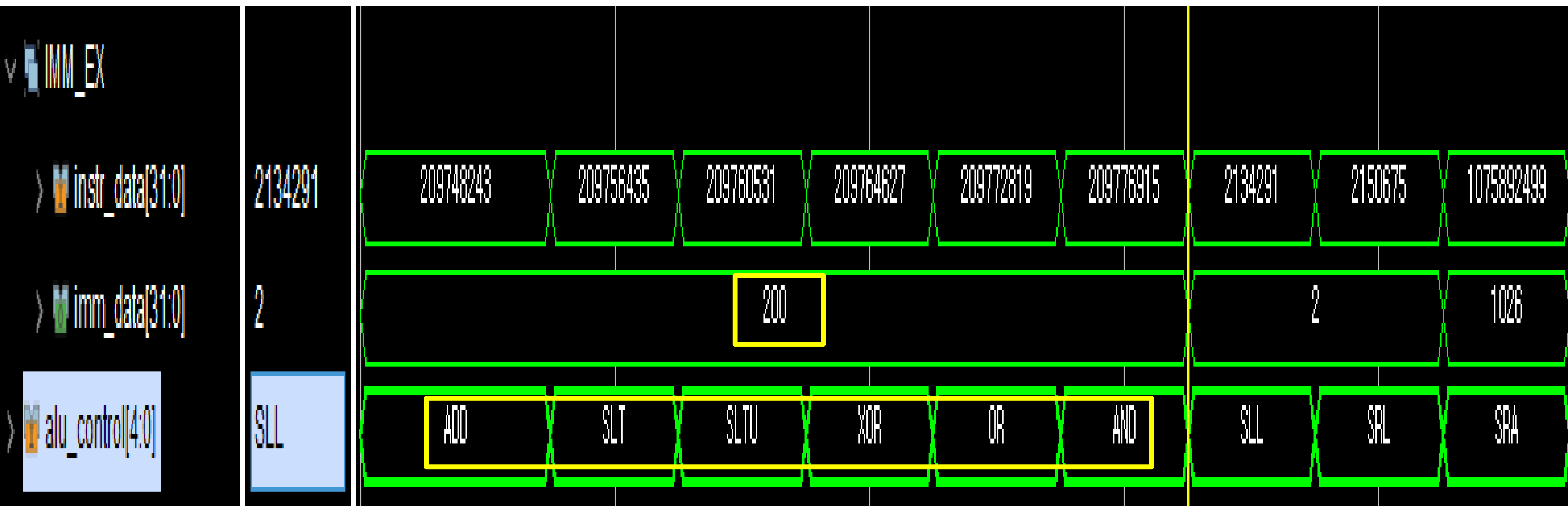
- simulation(Control Unit)

	ADD	SLL	SLT	SLTU	XOR	SRL	SRA	OR	AND
funct7[5]	x	0	x			0	1	x	
funct3	0	1	2	3	4	5		6	7



3. I-type

- simulation(imm_extender)

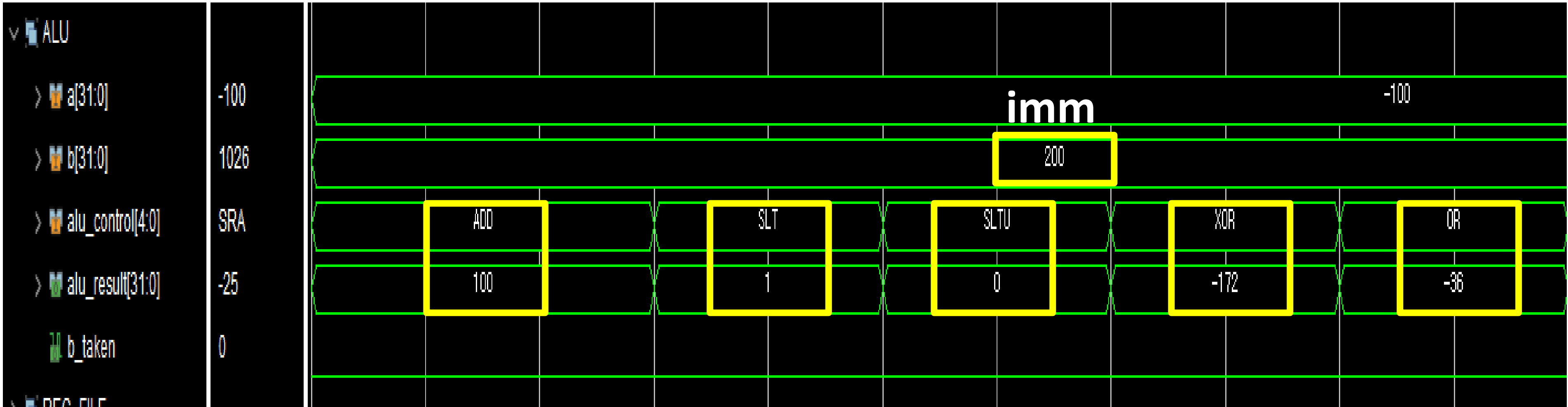


3. I-type

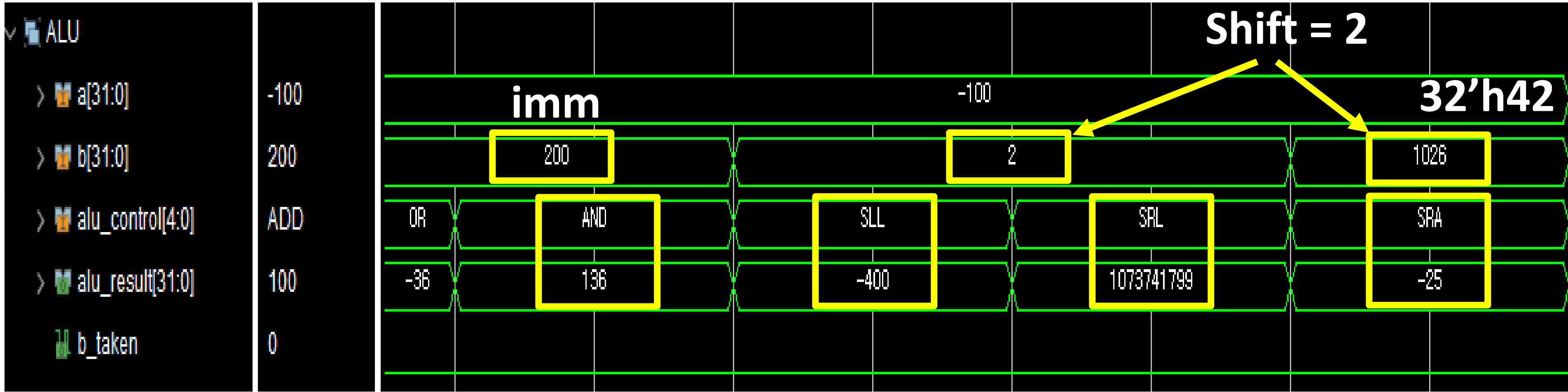
- simulation(ALU)

INSTR MEM		
PC	INSRT	Result(DEX)
0	ADDI	100
1	SLTI	1
2	SLTUI	0
3	XORI	-172
4	ORI	-36
5	ANDI	136
6	SRLI	1073741799
7	SRAI	-25

- ADD ~ SLTU



- AND ~ SRA



3. I-type

- simulation



3. S-type

	SB	SH	SW
funct3	0	1	2

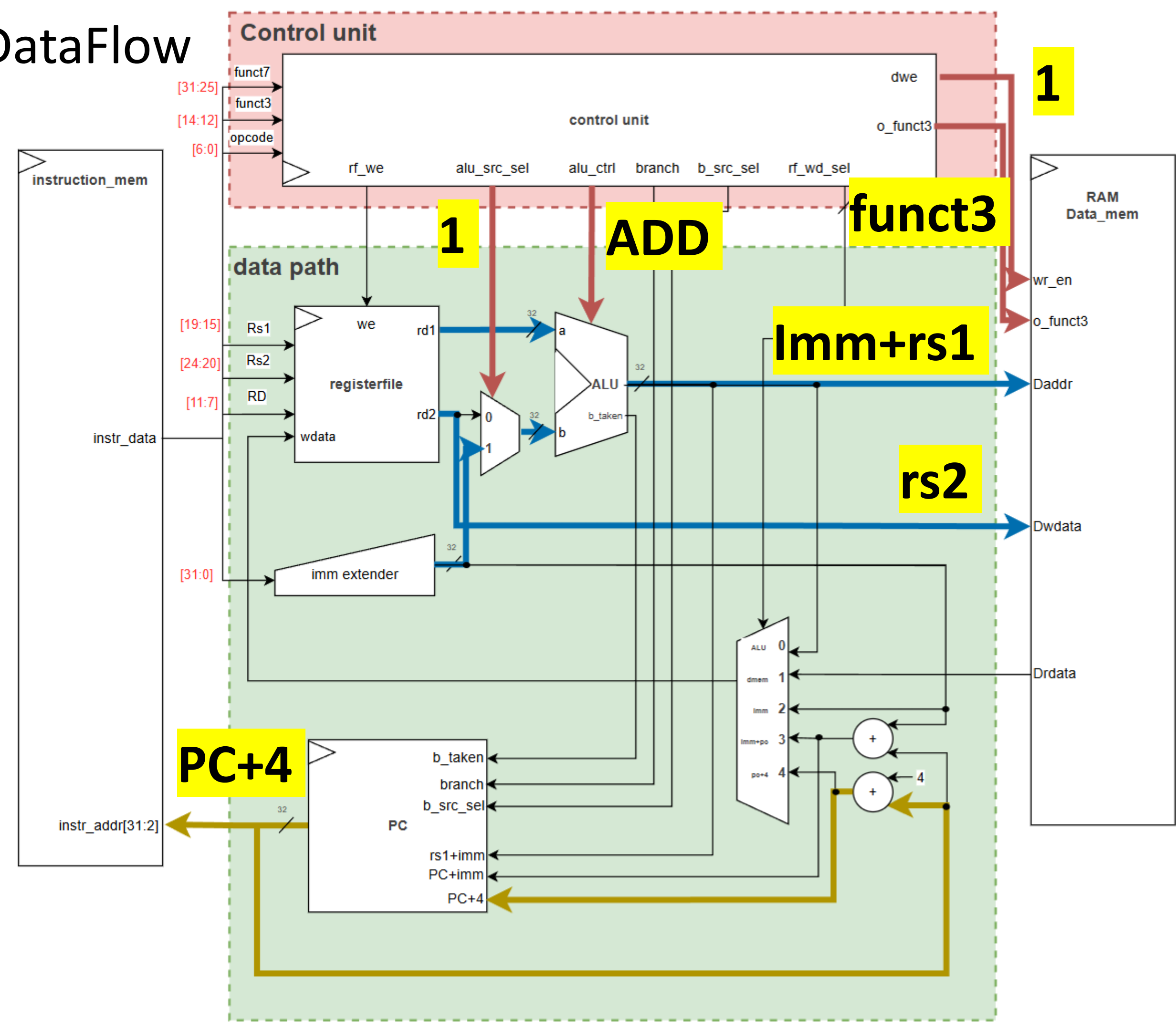
- S-type 명령어 : $\text{Dmem}[\text{rs1} + \text{imm}] = \text{rs2}$
- SB : byte addressing
- SH : half-word addressing
- SW: word addressing

S-type

Imm[11:5]	Rs2	Rs1	Funct3	Imm[4:0]	OPCODE(S-type)
-----------	-----	-----	--------	----------	----------------

3. S-type

• DataFlow

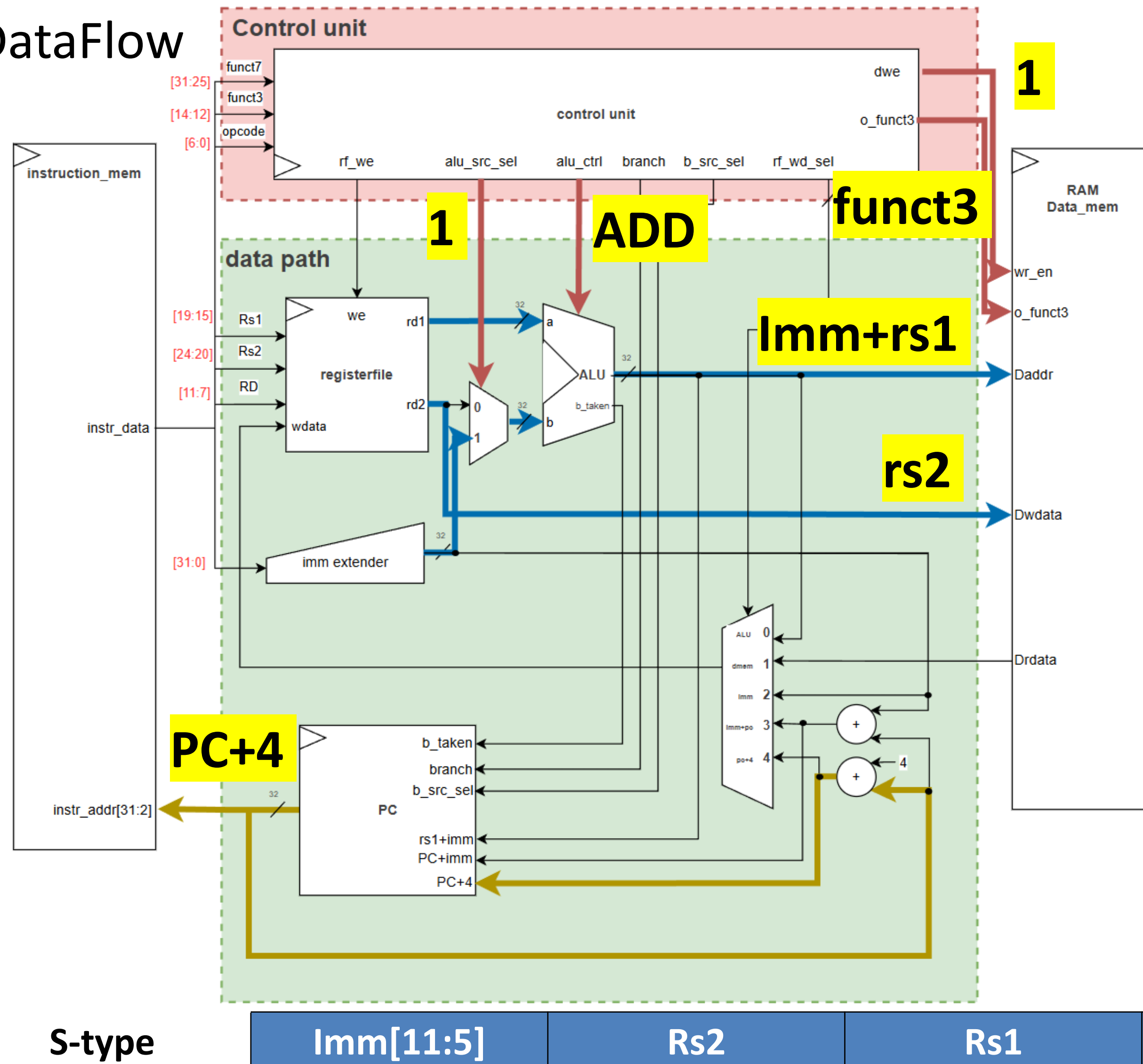


control_unit	
OUTPUT	S_type
rf_we	0
alu_src_sel	1
rf_wd_sel	0
alu_control [b_taken,ALU_ctrl]	ADD
b_src_sel	0
branch	0
o_funct3	funct3
dwe	1

S-type	Imm[11:5]	Rs2	Rs1	Funct3	Imm[4:0]	OPCODE(S-type)
--------	-----------	-----	-----	--------	----------	----------------

3. S-type

- DataFlow



DATA MEM				
INSRT	설계	Write 조건	Funct3	Addr scale
SB	SL word addressing	dwe = 1	3'h0	Byte
SH			3'h1	Half word
SW			3'h2	word

S-type

Imm[11:5]

Rs2

Rs1

Funct3

Imm[4:0]

OPCODE(S-type)

3. S-type

- simulation(시나리오)

Register File		
addr	name	value
28	Store Data	32'h44
29		32'h33
30		32'h22
31		32'h11

Dmem[Rs1+imm] = Rs2

Rs1 : 0

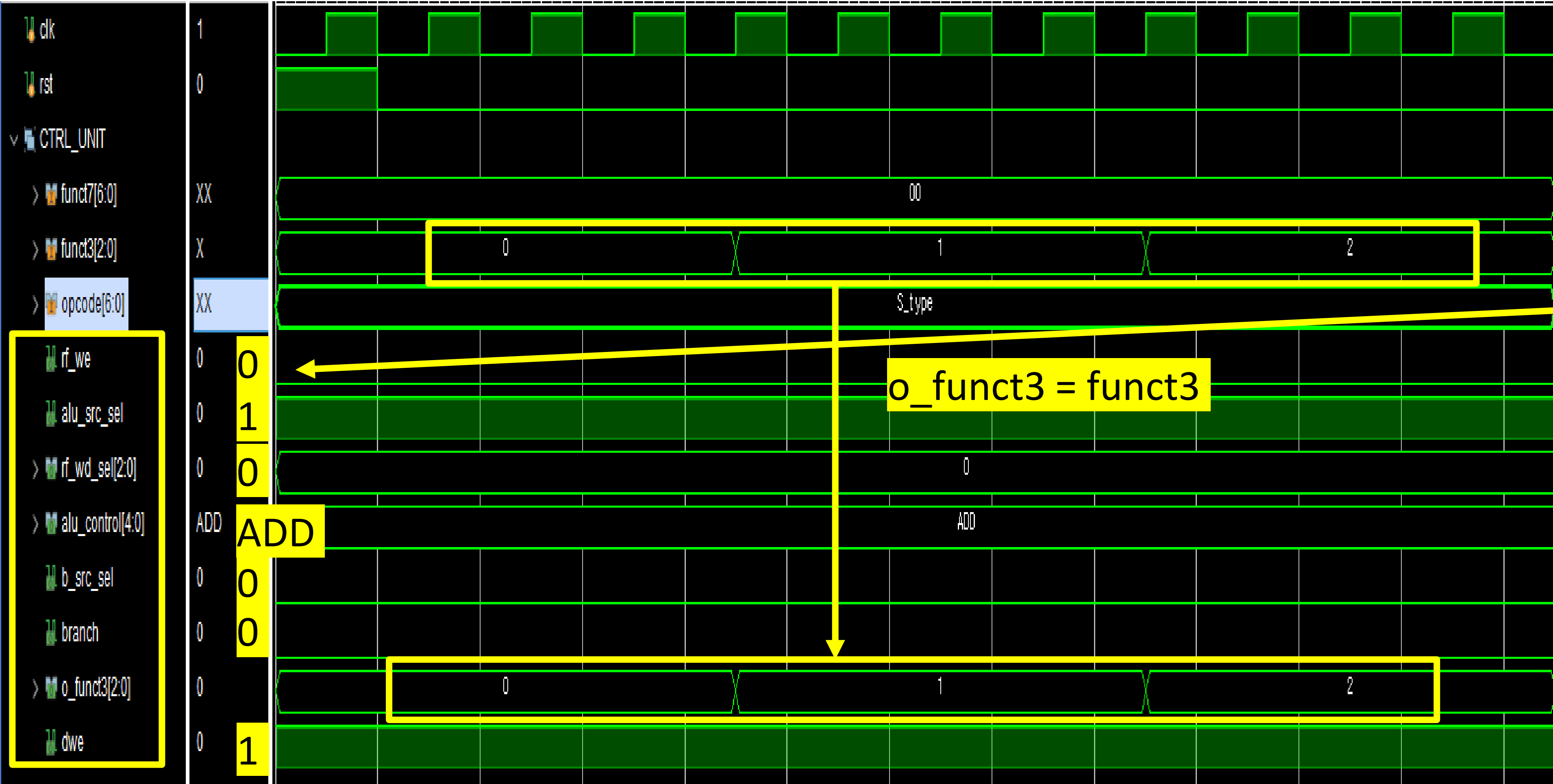
Imm : 0~3

Rs2 : x28 ~ x31

INSTR MEM			
PC	INSRT	(byte_addr 기준)	Dmem[0] (32'hx 초기화)
0	SB	Dmem[0+0] = 32'h44	xx_xx_xx_44
1		Dmem[0+1] = 32'h33	xx_xx_33_44
2		Dmem[0+2] = 32'h22	xx_22_33_44
3		Dmem[0+3] = 32'h11	11_22_33_44
4	SH	Dmem[0+0] = 32'h44	11_22_00_44
5		Dmem[0+1] = 32'h33	11_22_00_33
6		Dmem[0+2] = 32'h22	00_22_00_33
7		Dmem[0+3] = 32'h11	00_11_00_33
8	SW	Dmem[0+0] = 32'h44	00_00_00_44
9		Dmem[0+1] = 32'h33	00_00_00_33
10		Dmem[0+2] = 32'h22	00_00_00_22
11		Dmem[0+3] = 32'h11	00_00_00_11

3. S-type

- simulation(Control Uint)



control_unit	
OUTPUT	S_type
rf_we	0
alu_src_sel	1
rf_wd_sel	0
alu_control [b_taken,ALU_ctrl]	ADD
b_src_sel	0
branch	0
o_func3	funct3
dwe	1

3. S-type

$$Dmem[Rs1+imm] = Rs2$$

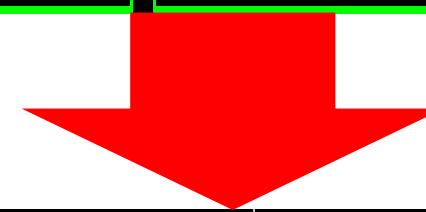
Rs1 : 0

Imm : 0~3

Rs2 : x28 ~ x31

- simulation(imm_extender & ALU)

IMM_EX		Data Values											
> instr_data[31:0]	X	29360163	30408867	31457571	32506275	29364259	30412963	31461667	32510371	29368355	30417059	31465763	32514467
> imm_data[31:0]	0	0	1	2	3	0	1	2	3	0	1	2	3

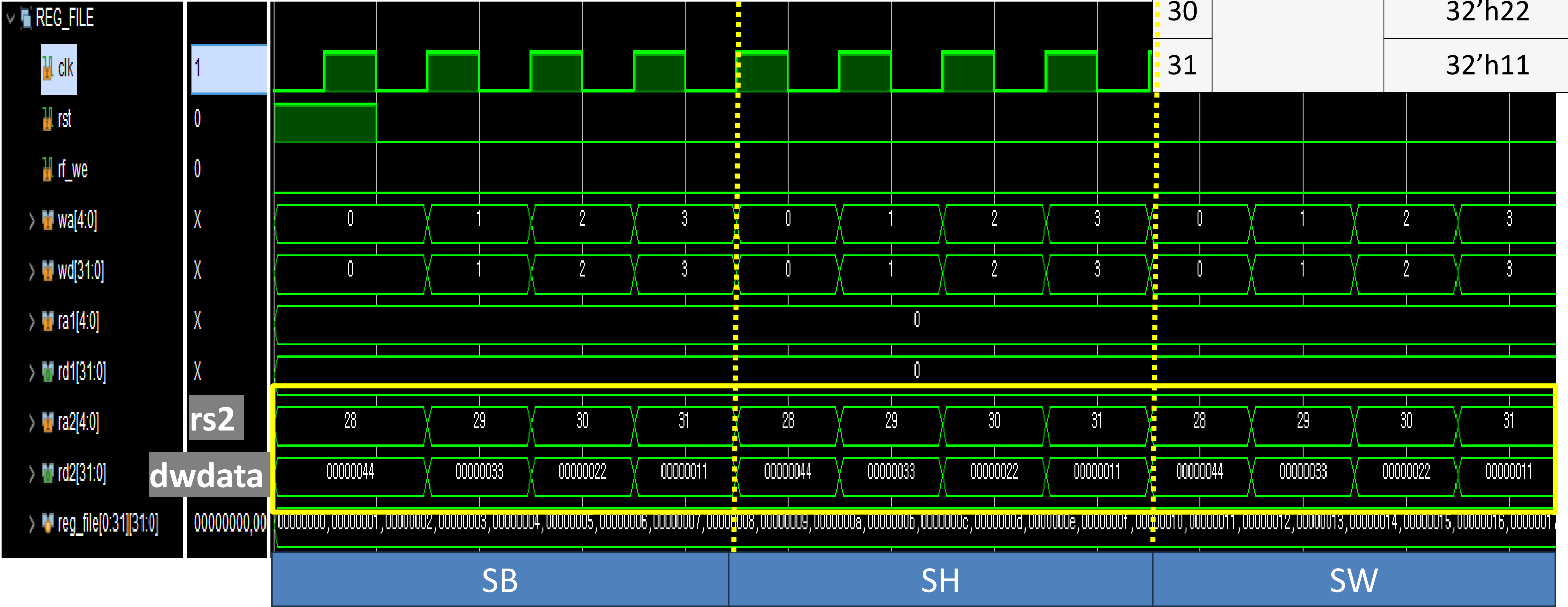


Field	Value	Field	Value	Field	Value
rs1	0	rs2	0	rs3	0
imm	0 1 2 3 0 1 2 3 0 1 2 3	imm2	0 1 2 3 0 1 2 3 0 1 2 3	imm3	0 1 2 3 0 1 2 3 0 1 2 3
ADD	ADD	ADD	ADD	ADD	ADD
daddr	0 1 2 3 0 1 2 3 0 1 2 3	daddr	0 1 2 3 0 1 2 3 0 1 2 3	daddr	0 1 2 3 0 1 2 3 0 1 2 3
b_taken	0	b_taken	0	b_taken	0
SB			SH		SW

3. S-type

- simulation(Register File)

Register File		
addr	name	value
28	Store Data	32'h44
29		32'h33
30		32'h22
31		32'h11



3. S-type

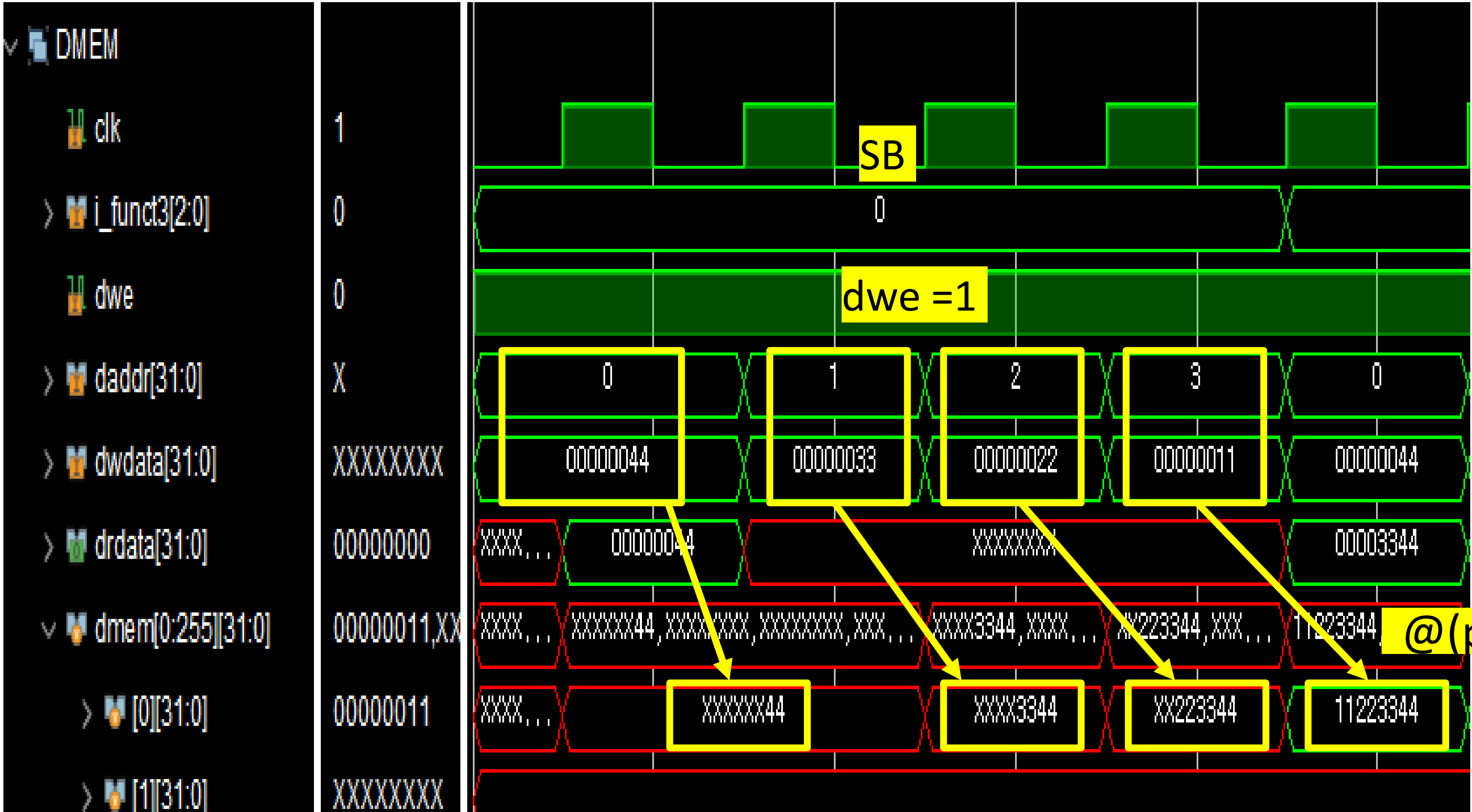
- simulation(Data mem)

$Dmem[Rs1+imm] = Rs2$

Rs1 : 0

Imm : 0~3

Rs2 : x28 ~ x31



INSTR MEM		
PC	INSRT	Dmem[0] = Result
0	SB	00_00_00_44
1		00_00_33_44
2		00_22_33_44
3		11_22_33_44

@(posedge clk)

3. S-type

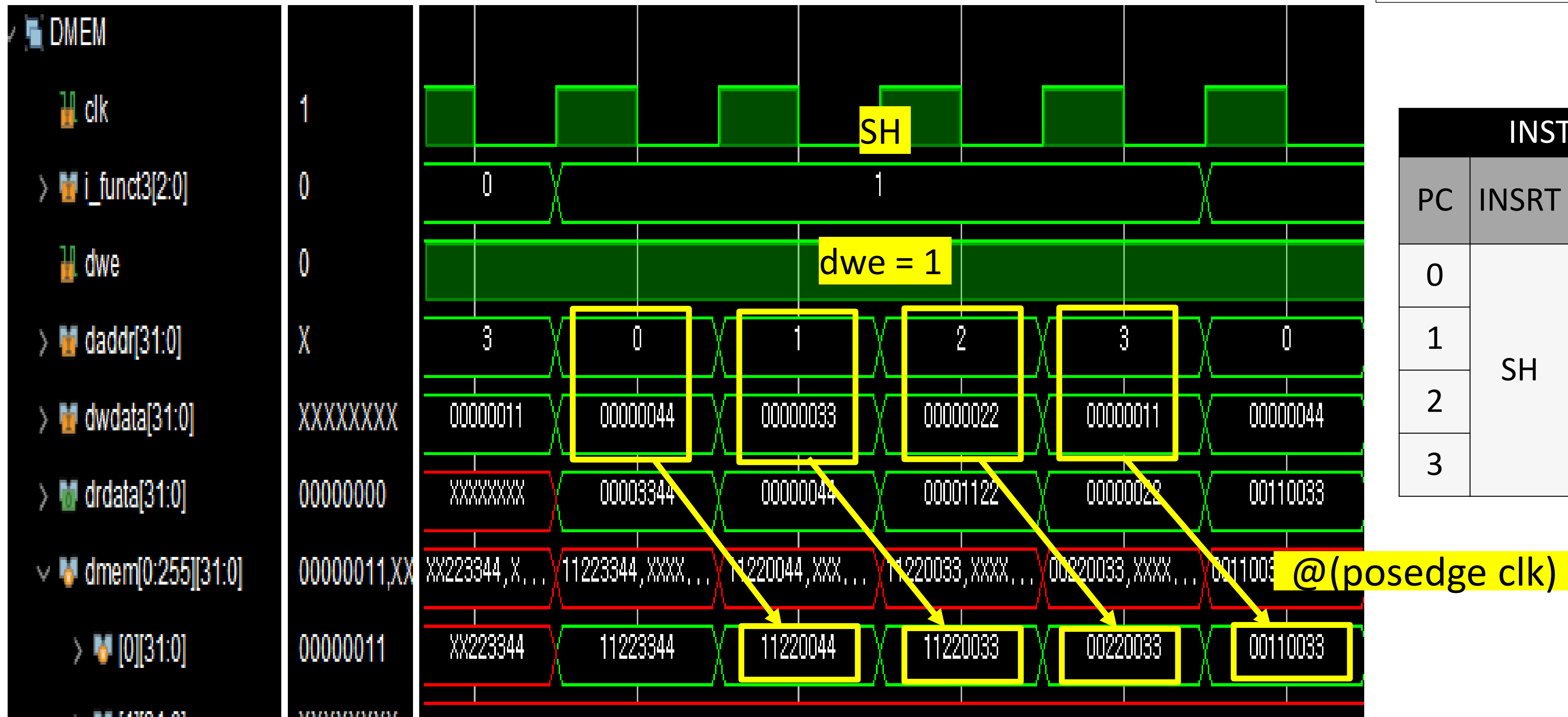
- simulation(Data mem)

$$Dmem[Rs1+imm] = Rs2$$

Rs1 : 0

Imm : 0~3

Rs2 : x28 ~ x31



3. S-type

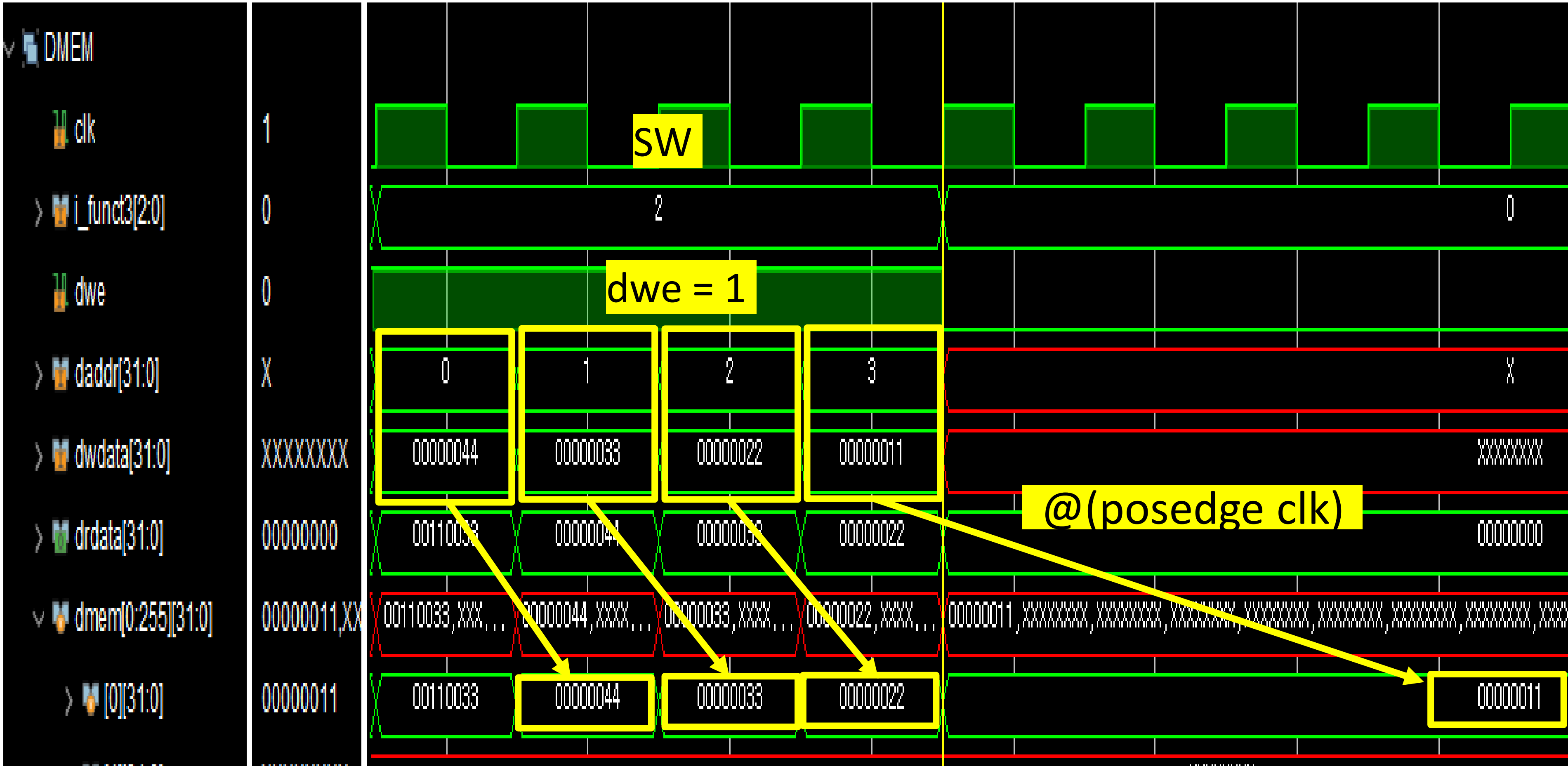
- simulation(Data mem)

$Dmem[Rs1+imm] = Rs2$

Rs1 : 0

Imm : 0~3

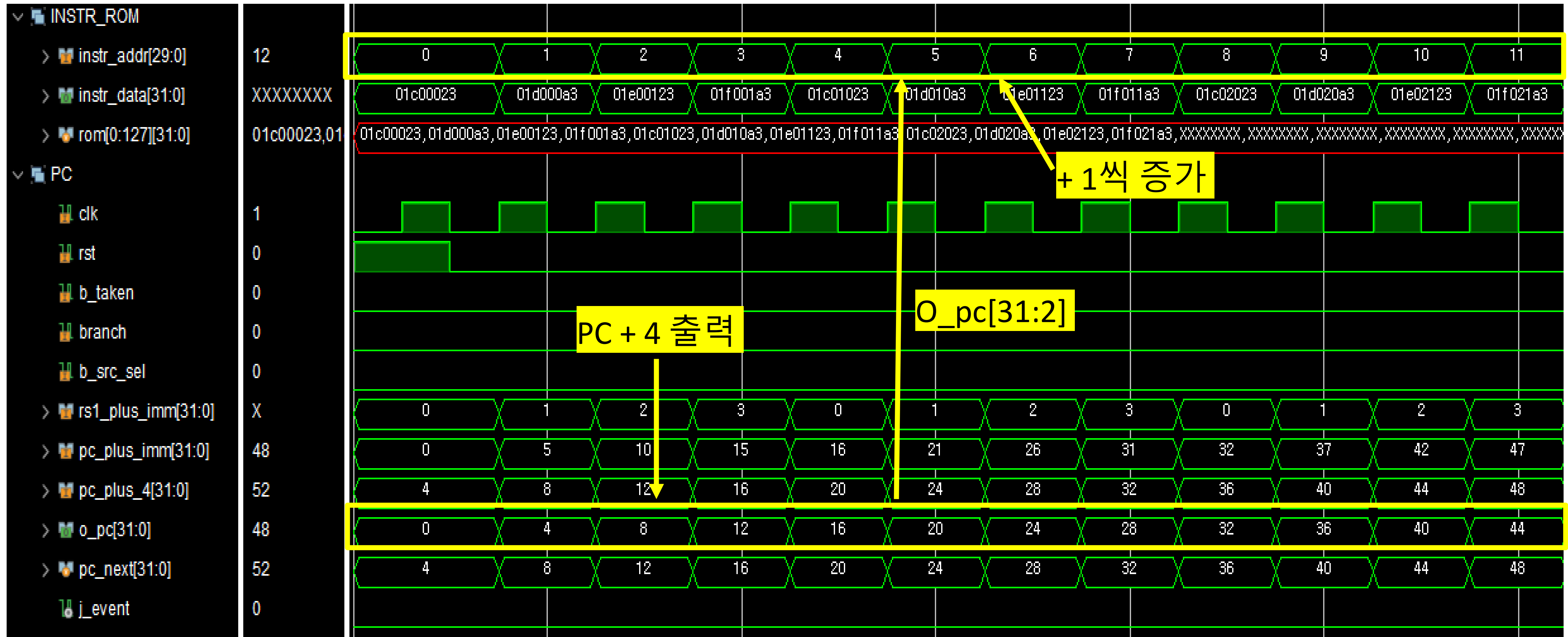
Rs2 : x28 ~ x31



INSTR MEM		
PC	INSRT	Dmem[0] = Result
0	SW	00_00_00_44
1		00_00_00_33
2		00_00_00_22
3		00_00_00_11

3. S-type

- simulation(PC)



3. IL-type

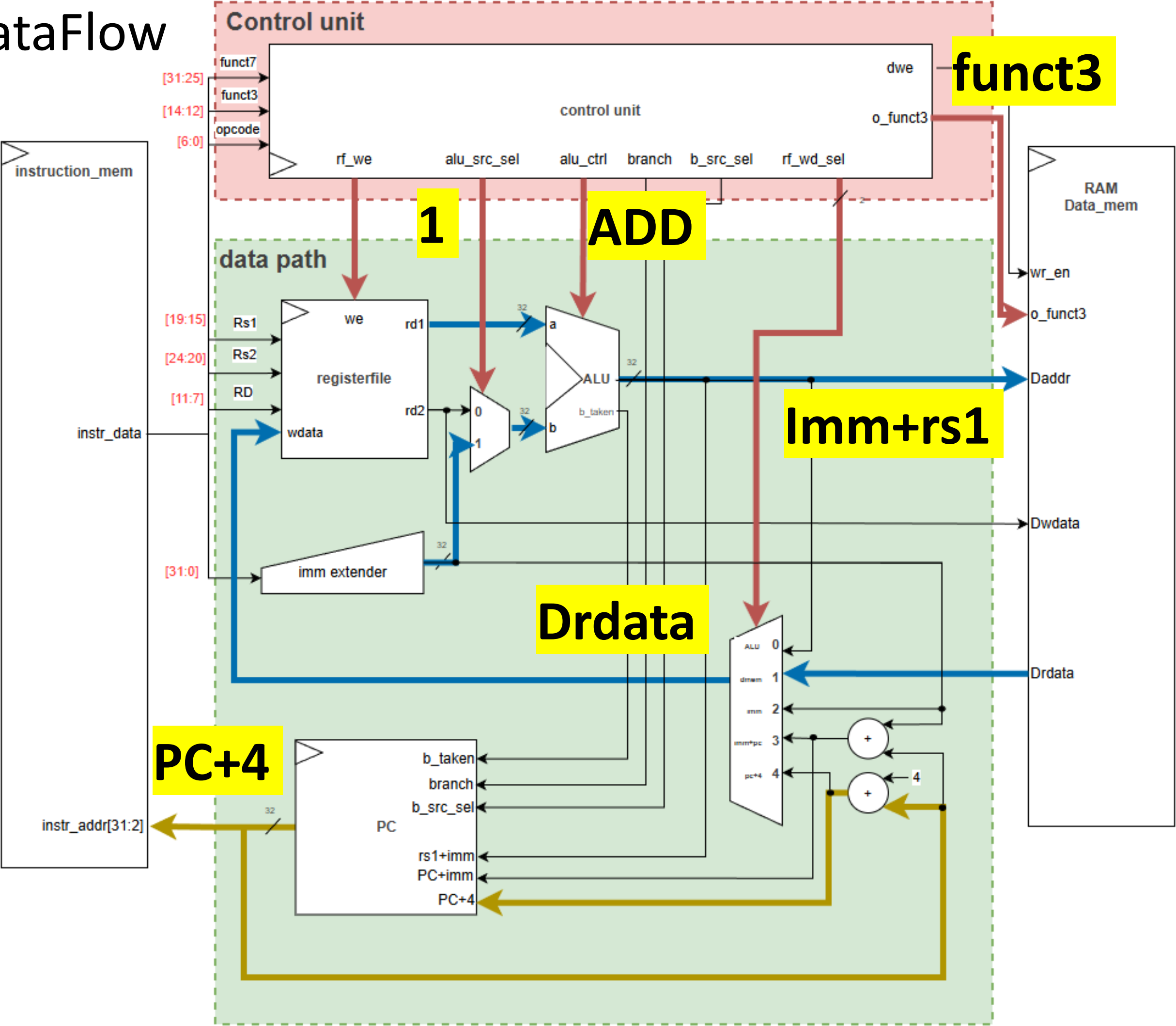
	LB	LH	LW	LBU	LHU
funct3	0	1	2	3	4
signed	S			U	

- IL-type 명령어 : $rd = Dmem[rs1 + imm]$
- I-type의 Load 명령어는 I-type의 산술 명령어와 기능적 분리하여 설계

IL-type	Imm[11:0]	Rs1	Funct3	Rd	OPCODE(IL-type)
---------	-----------	-----	--------	----	-----------------

3. IL-type

• DataFlow

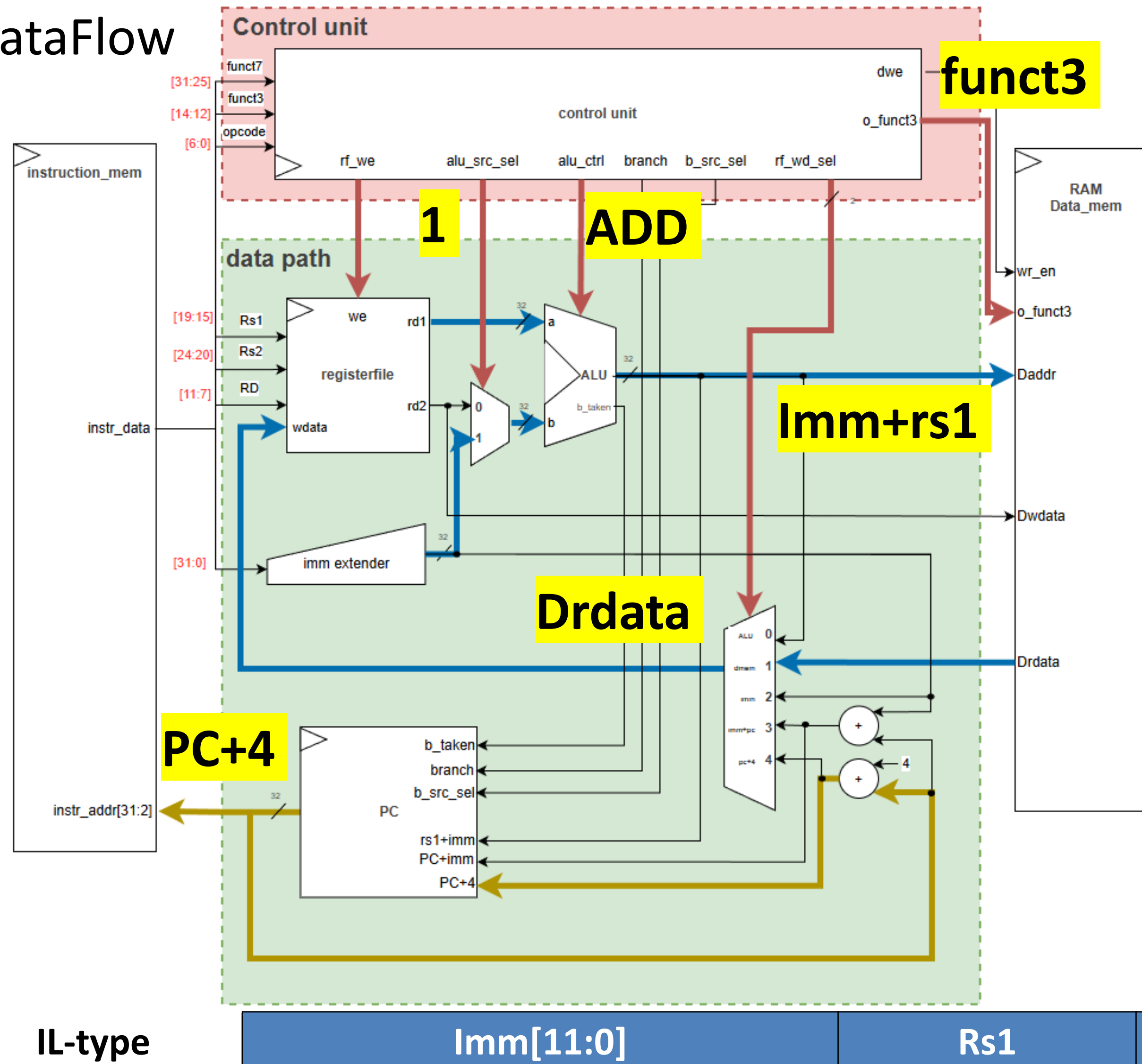


control_unit	
OUTPUT	IL_type
rf_we	1
alu_src_sel	1
rf_wd_sel	1
alu_control [b_taken,ALU_ctrl]	ADD
b_src_sel	0
branch	0
o_funct3	funct3
dwe	0

IL-type	Imm[11:0]	Rs1	Funct3	Rd	OPCODE(I-type)
---------	-----------	-----	--------	----	----------------

3. IL-type

- DataFlow



DATA MEM				
INSRT	설계	Funct3	Addr scale	Signed
LB	CL word addressing	3'h0	Byte	Signed
LH		3'h1	Half word	Signed
LW		3'h2	word	Signed
LBU		3'h3	Byte	Unsigned
LHU		3'h4	Half word	Unsigned

IL-type	Imm[11:0]	Rs1	Funct3	Rd	OPCODE(I-type)
---------	-----------	-----	--------	----	----------------

3. IL-type

• simulation(시나리오)

Rd = Dmem[Rs1+imm]

Rs1 : 0

Imm : 0~3

Rd : x1

DATA MEM	
addr	value
0	32'h04_f3_f2_01



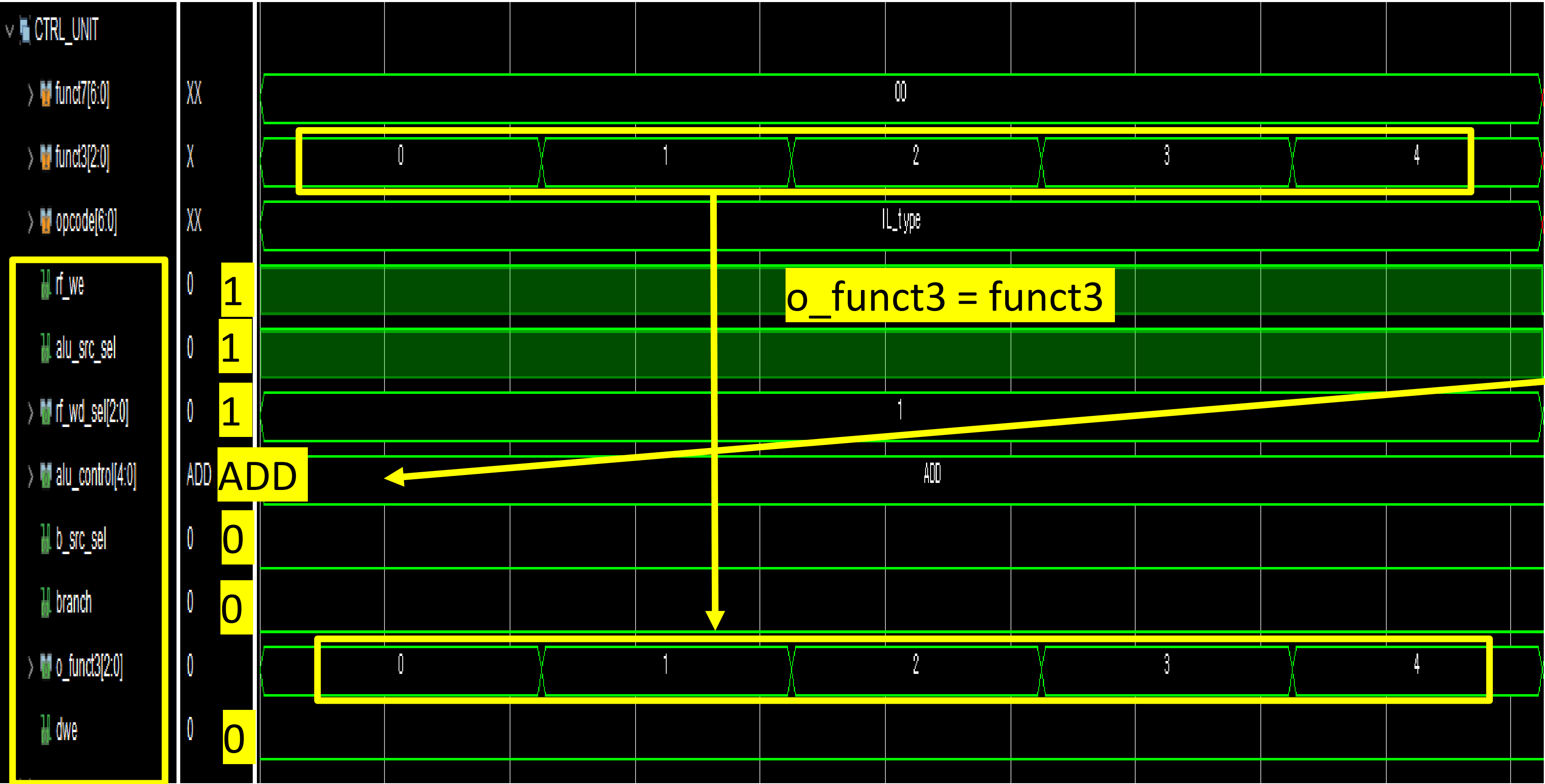
Register File	
x1	load data

INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
0	LB	Dmem[0+0] = 32'h01	00_00_00_01
1		Dmem[0+1] = 32'hf2	ff_ff_ff_f2
2		Dmem[0+2] = 32'hf3	ff_ff_ff_f3
3		Dmem[0+3] = 32'h04	00_00_00_04
4	LH	Dmem[0+0] = 32'h01	ff_ff_f2_01
5		Dmem[0+1] = 32'hf2	
6		Dmem[0+2] = 32'hf3	00_00_04_f3
7		Dmem[0+3] = 32'h04	
8	LW	Dmem[0+0] = 32'h01	04_f3_f2_01
9		Dmem[0+1] = 32'hf2	
10		Dmem[0+2] = 32'hf3	
11		Dmem[0+3] = 32'h04	

INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
12	LBU	Dmem[0+0] = 32'h01	00_00_00_01
13		Dmem[0+1] = 32'hf2	00_00_00_f2
14		Dmem[0+2] = 32'hf3	00_00_00_f3
15		Dmem[0+3] = 32'h04	00_00_00_04
16	LHU	Dmem[0+0] = 32'h01	00_00_f2_01
17		Dmem[0+1] = 32'hf2	
18		Dmem[0+2] = 32'hf3	00_00_04_f3
19		Dmem[0+3] = 32'h04	

3. IL-type

- simulation(ALU)



control_unit	
OUTPUT	IL_type
rf_we	1
alu_src_sel	1
rf_wd_sel	1
alu_control [b_taken,ALU_ctrl]	ADD
b_src_sel	0
branch	0
o_func3	funct3
dwe	0

3. IL-type

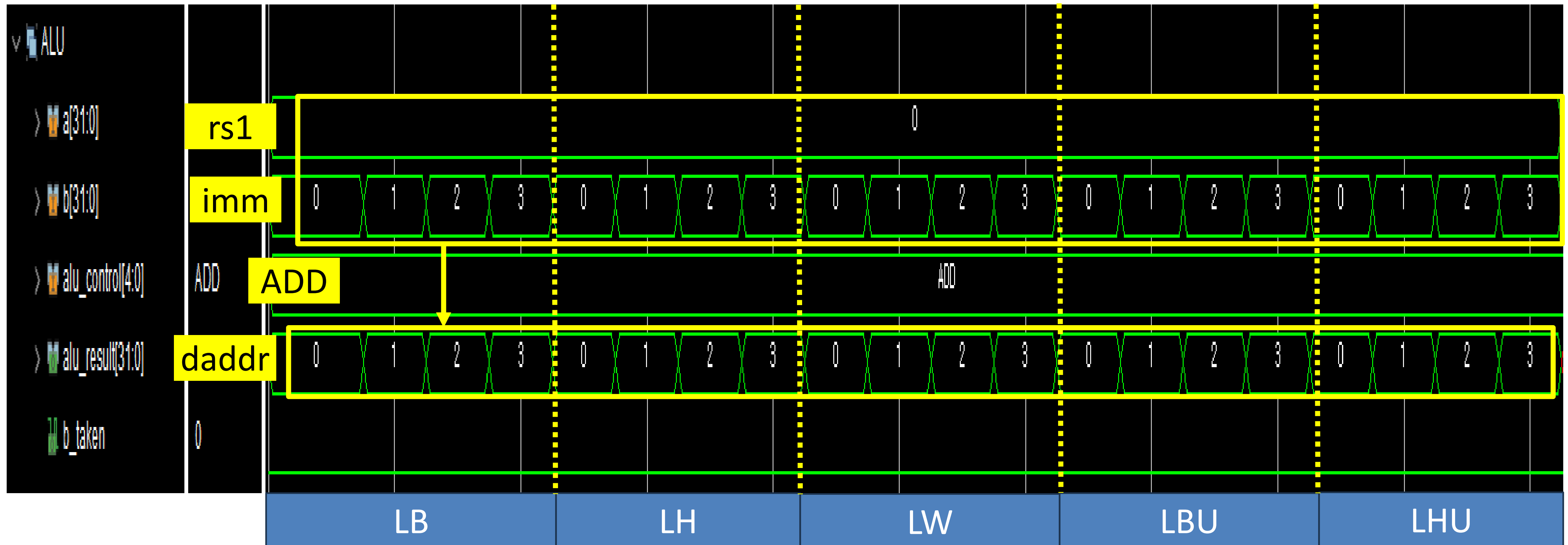
- simulation(ALU)

$Rd = Dmem[Rs1+imm]$

Rs1 : 0

Imm : 0~3

Rd : x1



3. IL-type

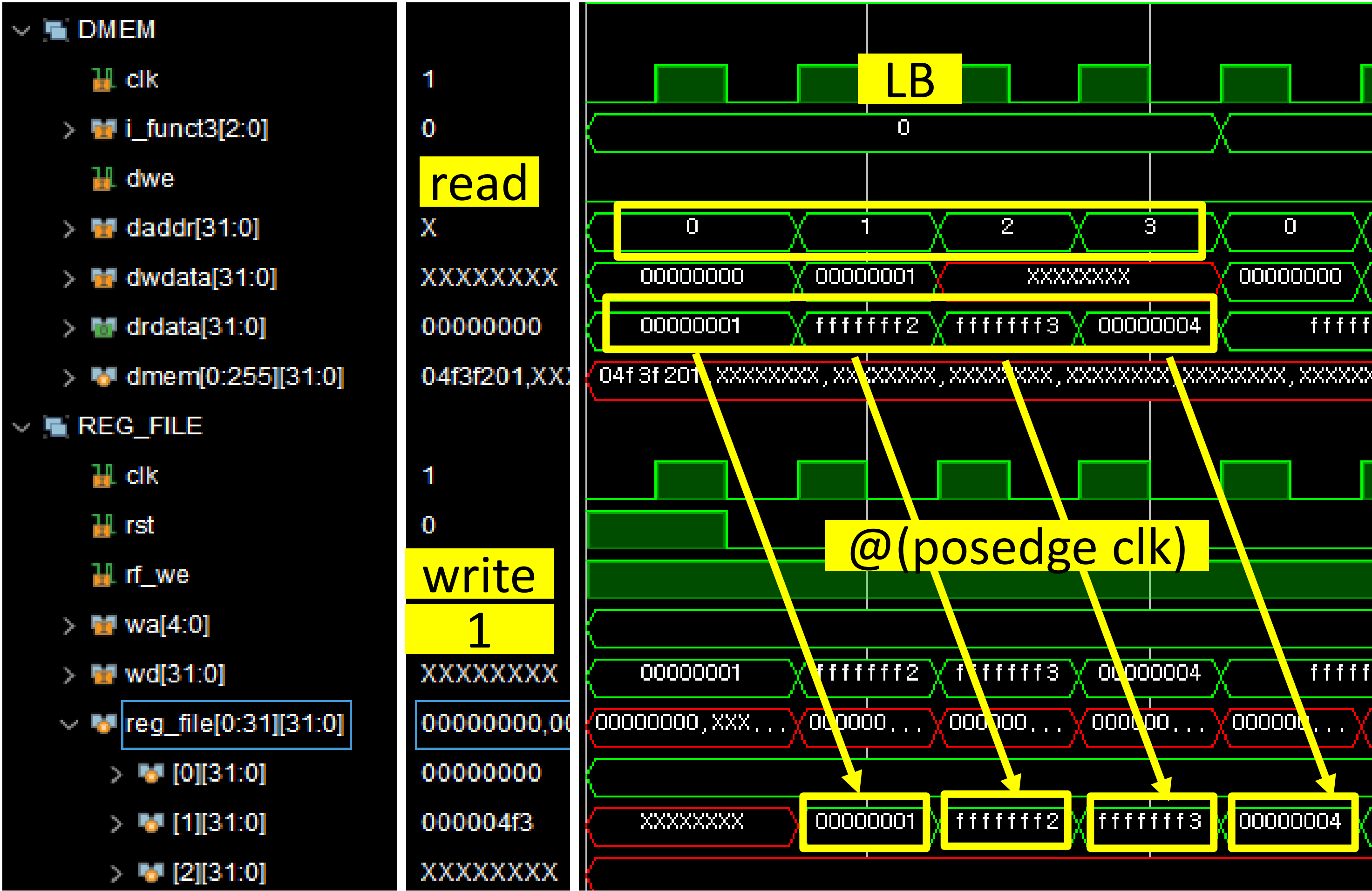
- simulation(Data mem & register file)

$Rd = Dmem[Rs1+imm]$

Rs1 : 0

Imm : 0~3

Rd : x1



INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
0	LB	Dmem[0+0] = 32'h01	00_00_00_01
1		Dmem[0+1] = 32'hf2	ff_ff_ff_f2
2		Dmem[0+2] = 32'hf3	ff_ff_ff_f3
3		Dmem[0+3] = 32'h04	00_00_00_04

3. IL-type

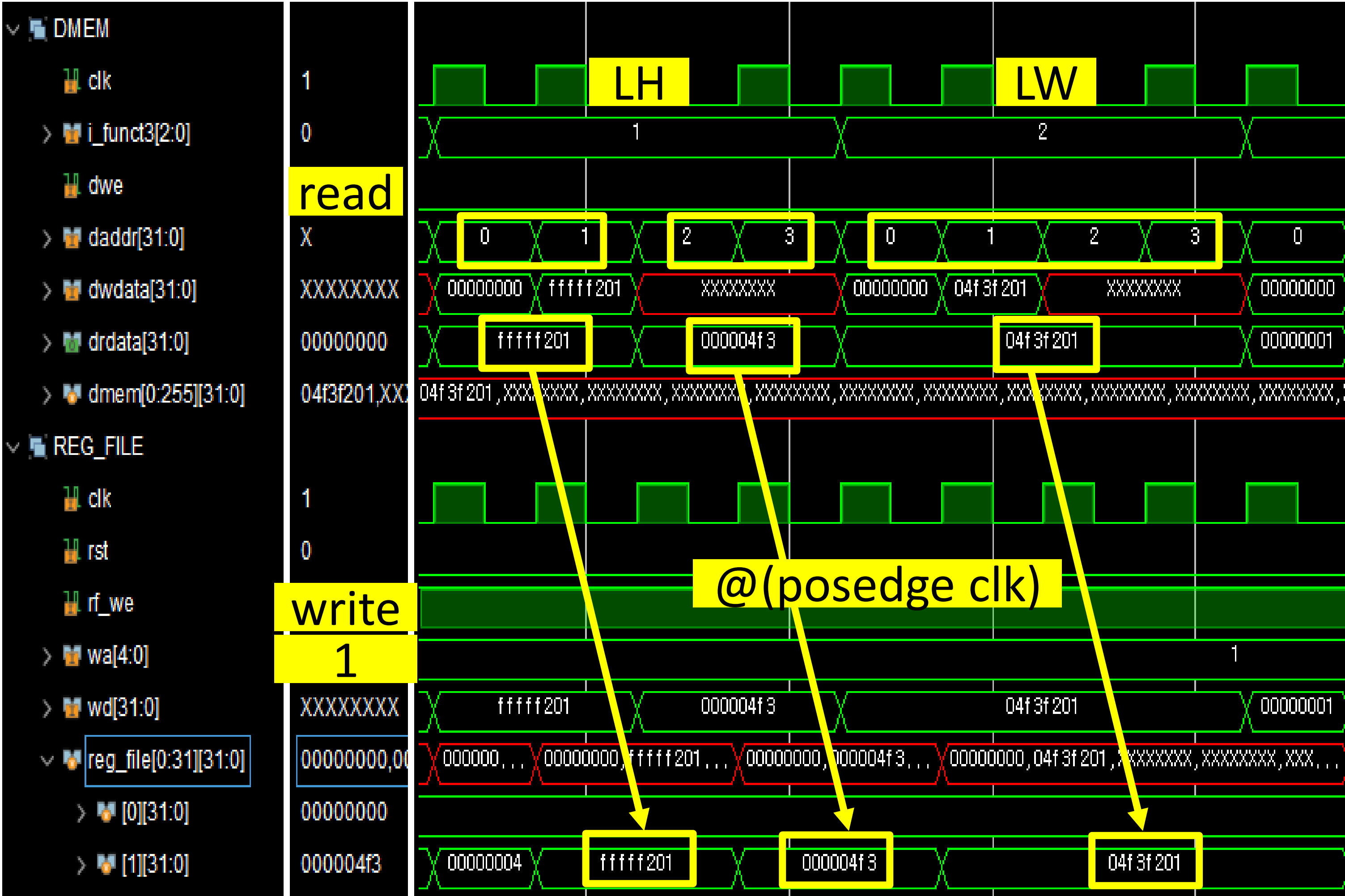
- simulation(Data mem & register file)

Rd = Dmem[Rs1+imm]

Rs1 : 0

Imm : 0~3

Rd : x1



INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
4	LH	Dmem[0+0] = 32'h01	ff_ff_f2_01
5		Dmem[0+1] = 32'hf2	
6		Dmem[0+2] = 32'hf3	00_00_04_f3
7		Dmem[0+3] = 32'h04	
8	LW	Dmem[0+0] = 32'h01	04_f3_f2_01
9		Dmem[0+1] = 32'hf2	
10		Dmem[0+2] = 32'hf3	
11		Dmem[0+3] = 32'h04	

3. IL-type

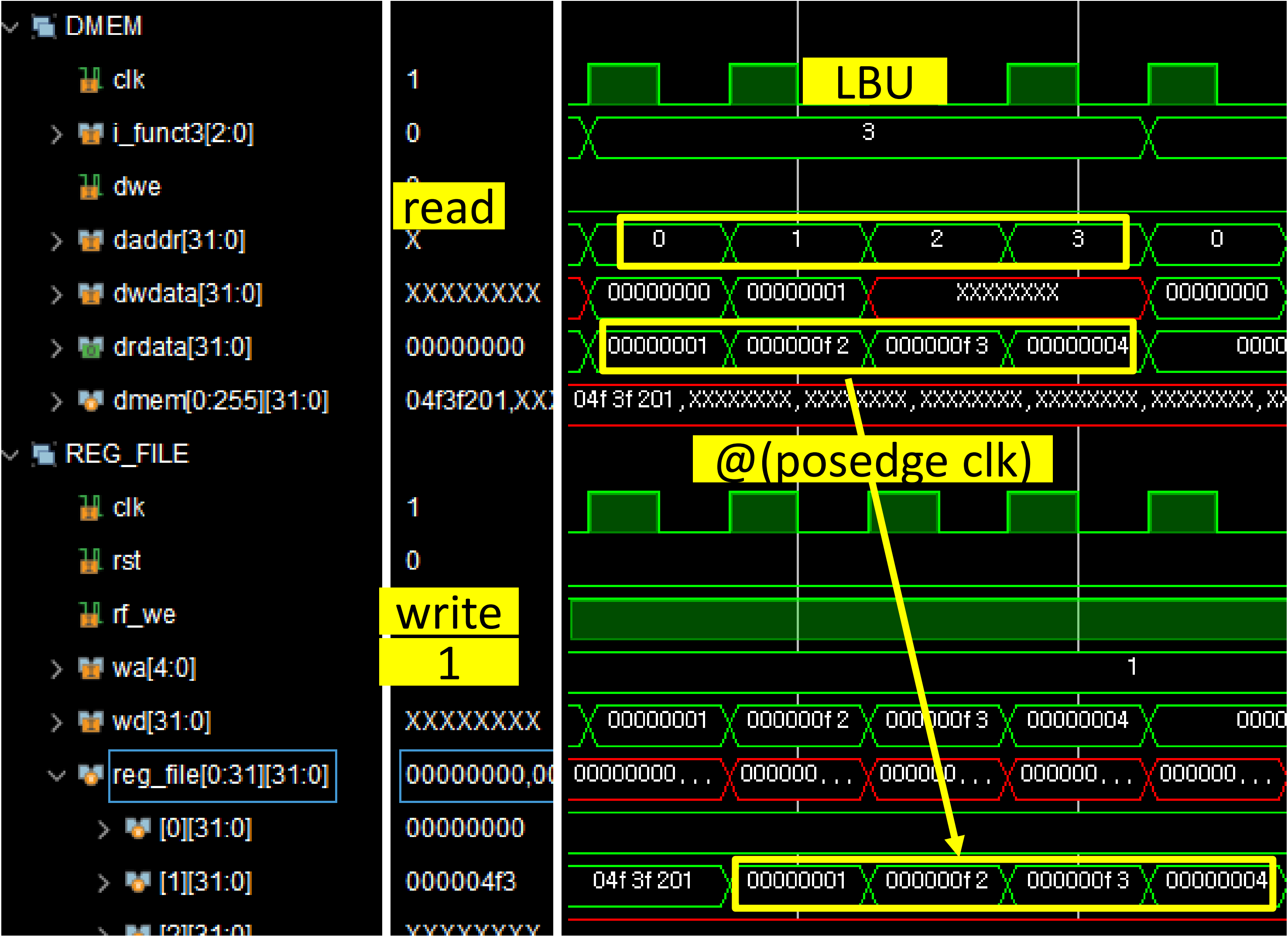
- simulation(Data mem & register file)

Rd = Dmem[Rs1+imm]

Rs1 : 0

Imm : 0~3

Rd : x1



INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
12	LBU	Dmem[0+0] = 32'h01	00_00_00_01
13		Dmem[0+1] = 32'hf2	00_00_00_f2
14		Dmem[0+2] = 32'hf3	00_00_00_f3
15		Dmem[0+3] = 32'h04	00_00_00_04

3. IL-type

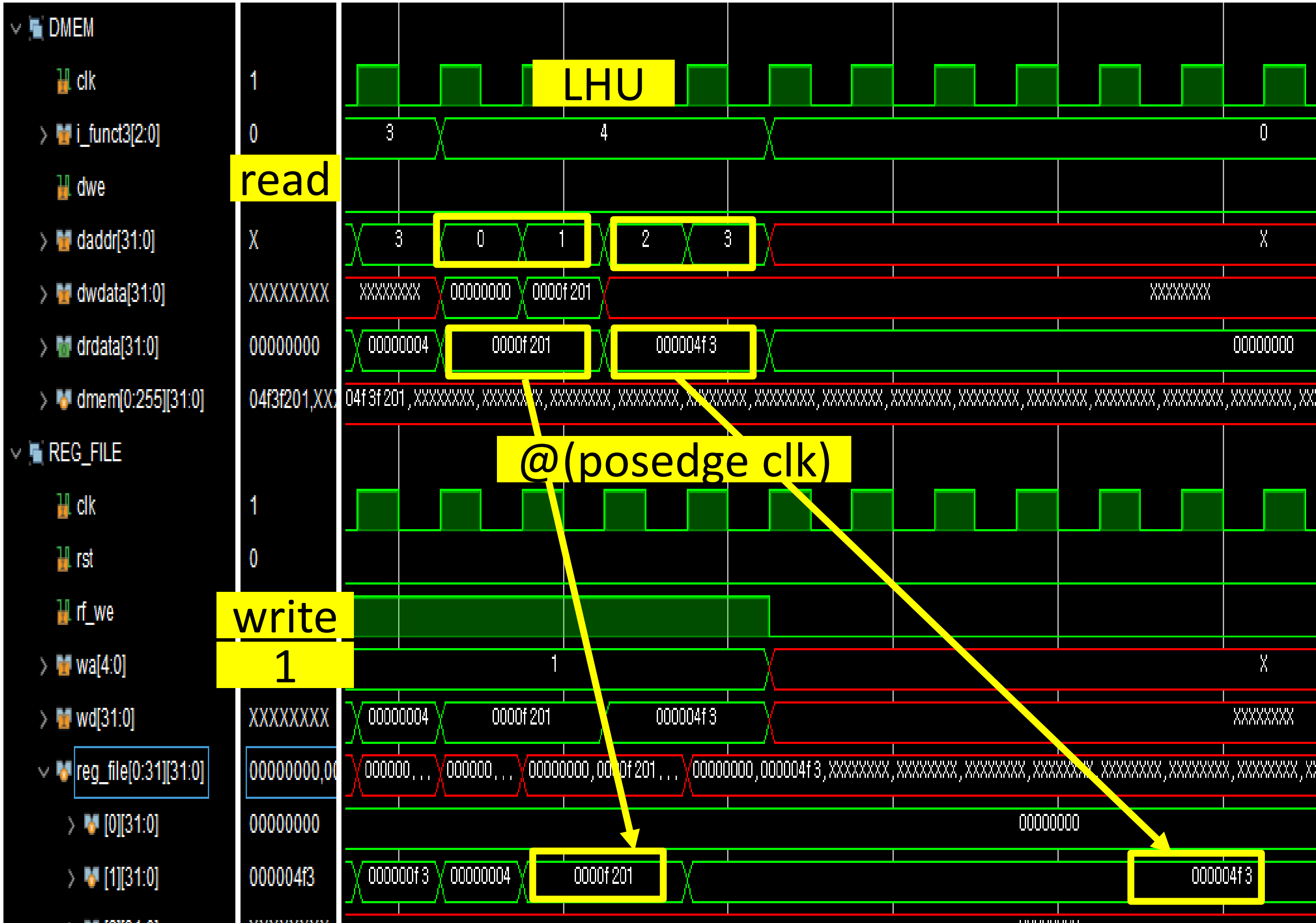
- simulation(Data mem & register file)

Rd = Dmem[Rs1+imm]

Rs1 : 0

Imm : 0~3

Rd : x1



INSTR MEM			
PC	INSRT	(byte_addr 기준)	reg file[x1]
16	LHU	Dmem[0+0] = 32'h01	00_00_f2_01
17		Dmem[0+1] = 32'hf2	
18		Dmem[0+2] = 32'hf3	00_00_04_f3
19		Dmem[0+3] = 32'h04	

3. B-type

	BEQ	BNE	BLT	BGE	BLTU	BGEU
Funct3	0	1	4	5	6	7
	$a == b$	$a != b$	$a < b$	$a \geq b$	$a < b$	$a \geq b$

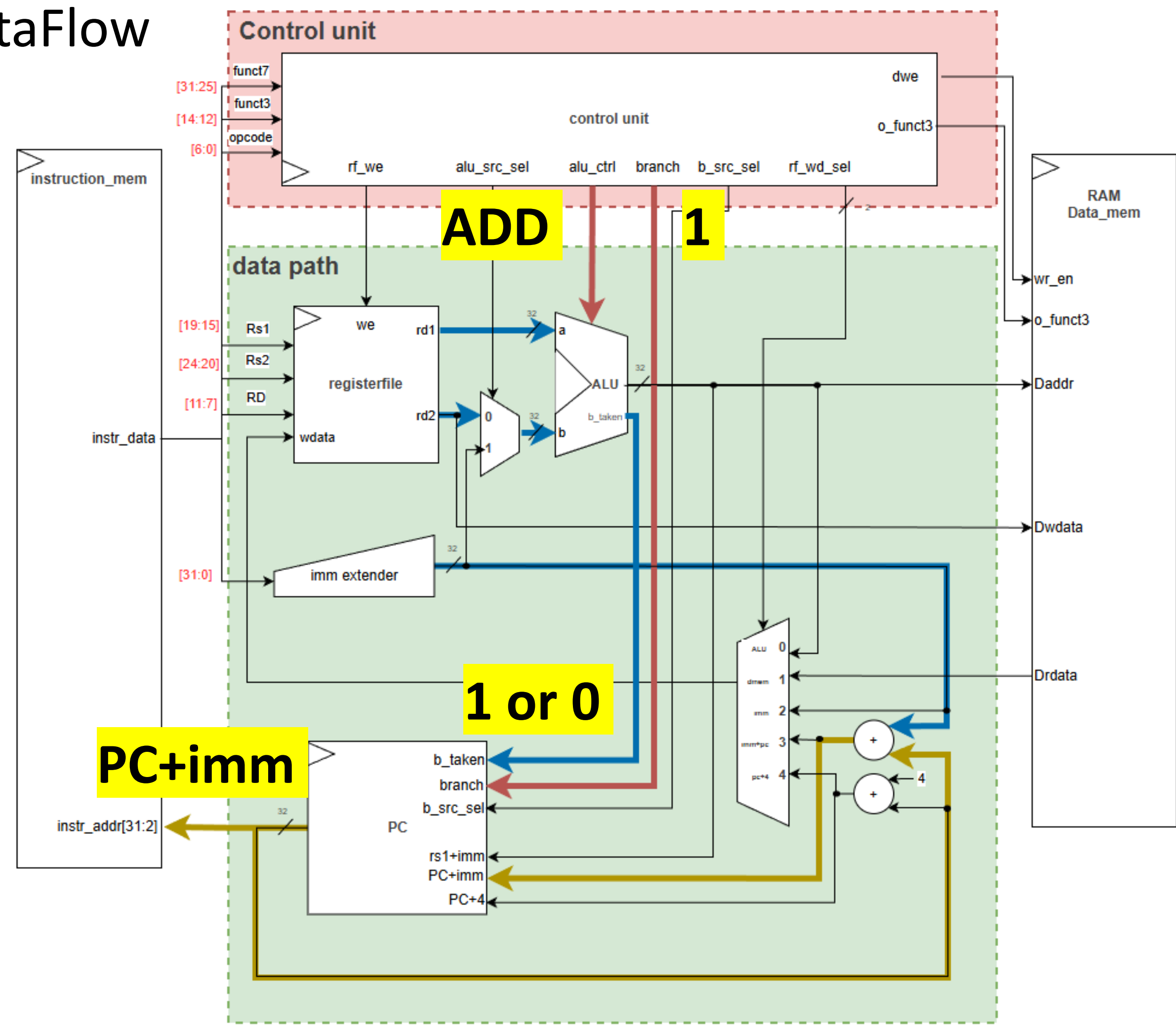
- B-type은 rs1, rs2에 대해 비교 연산자를 사용
- 참일 경우, PC는 $PC + imm$ 으로 이동(Jump)

B-type

Imm[12,10:5]	Rs2	Rs1	Funct3	Imm[4:1,11]	OPCODE(B-type)
--------------	-----	-----	--------	-------------	----------------

3. B-type

• DataFlow



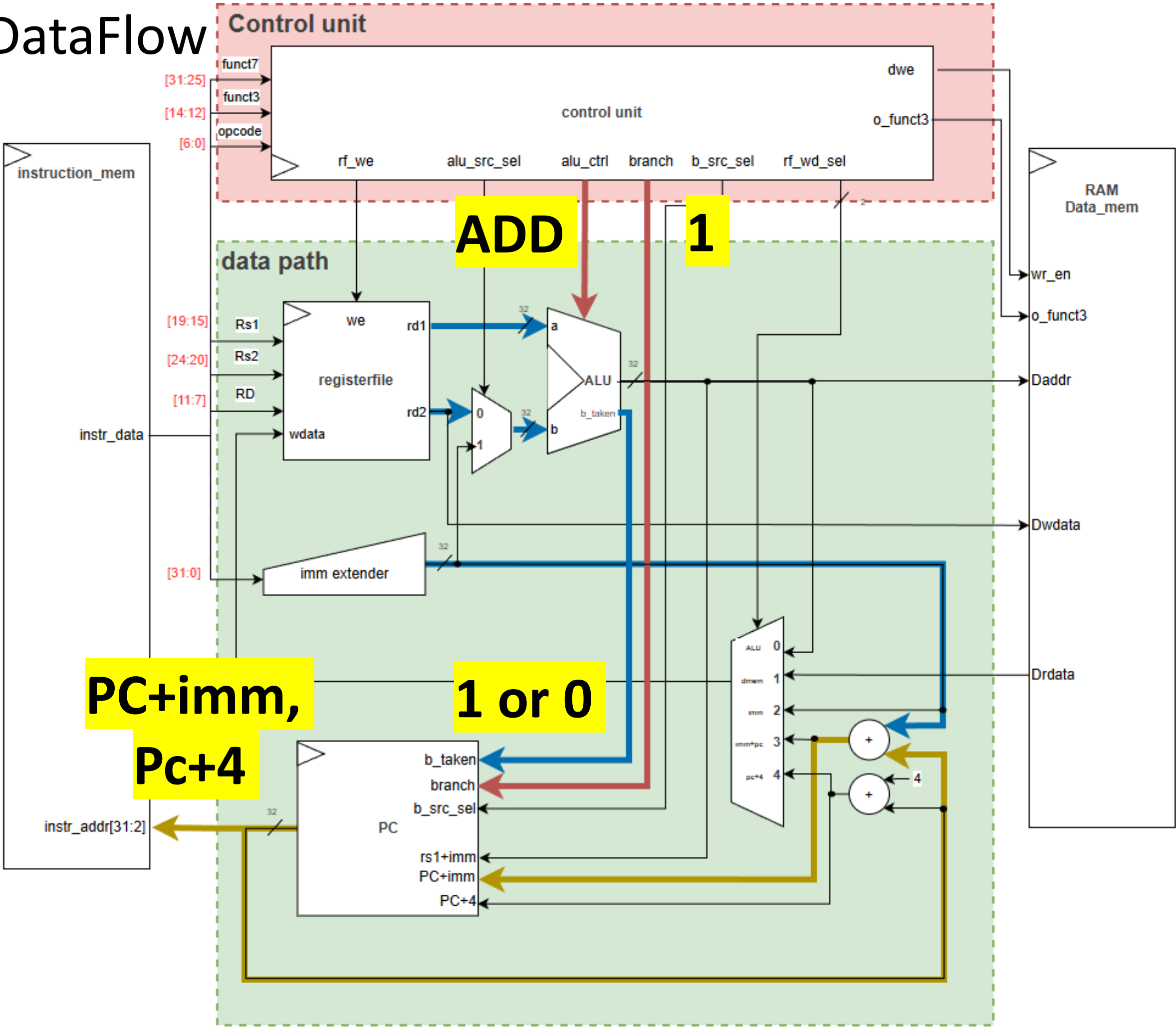
control_unit	
OUTPUT	B_type
rf_we	0
alu_src_sel	0
rf_wd_sel	0
alu_control [b_taken,ALU_ctrl]	1,0,funct3
b_src_sel	0
branch	1
o_funct3	0
dwe	0

PC		
Case		OUTPUT
j_event	b_src_sel	(pc_next)
0	x	PC + 4
1	0	PC + imm
1	1	rs1 + imm

B-type	Imm[12,10:5]	Rs2	Rs1	Funct3	Imm[4:1,11]	OPCODE(B-type)
--------	--------------	-----	-----	--------	-------------	----------------

3. B-type

• DataFlow



ALU							
Ctrl		BEQ	BNE	BLT	BGE	BLTU	BGEU
[4]	branch	1					
[3]	Funct7 [5]	0					
[2:0]	Funct3	0	1	4	5	6	7
OUTPUT							
ALU		a == b	a != b	a < b	a >= b	a < b	a >= b
signed		S				U	
b_taken		T/F					

PC		
Case		OUTPUT
j_event	b_src_sel	(pc_next)
0	x	PC + 4
1	0	PC + imm
1	1	rs1 + imm

B-type

Imm[12,10:5]	Rs2	Rs1	Funct3	Imm[4:1,11]	OPCODE(B-type)
--------------	-----	-----	--------	-------------	----------------

3. B-type

- simulation(시나리오)

if(rs1 비교 rs2) PC += imm

register file	
addr	value
x0	0
x1	-1
x2	SUB 값

True case

PC += 2

PC += 6

PC += -2

PC += 8

INSTR MEM						
PC	imm	INSR T	ALU		T/F signed	
0	2*4	BEQ	-1 == -1		S	
2		BNE	0 != -1			
4		BLT	-1 < 0			
6	6*4	BGE	0 >= -1	>	T	
8			-1 >= -1	==		
14	-2*4	BLTU	0 < -1		U	
12		BGE U	-1 >= 0	>		
10	8*4		-1 >= -1	==		

false case

PC += 30*4

INSTR MEM						
PC	imm	INSRT	ALU		T/F signed	
18	30*4	BEQ	-1 == 0		S	
19		BNE	0 != 0			
20		BLT	0 < -1	<		
21			0 < 0	==	F	
22		BGE	-1 >= 0			
23		BLTU	-1 < 0	<		
24			0 < 0	==	U	
25		BGEU	0 >= -1			
26		SUB	x2 = (-1) - (-1)			

PC += 30*4

BEQ	BNE	BLT	BGE	BLTU	BGEU
1					
0					
0	1	4	5	6	7

- simulation(Control Unit)

CTRL_UNIT

- func7[6:0]
- func3[2:0]
- opcode[6:0]
- rf_we
- alu_src_sel
- rf_wd_sel[2:0]
- alu_control[4:0]
- b_src_sel
- branch
- o_func3[2:0]
- dwe

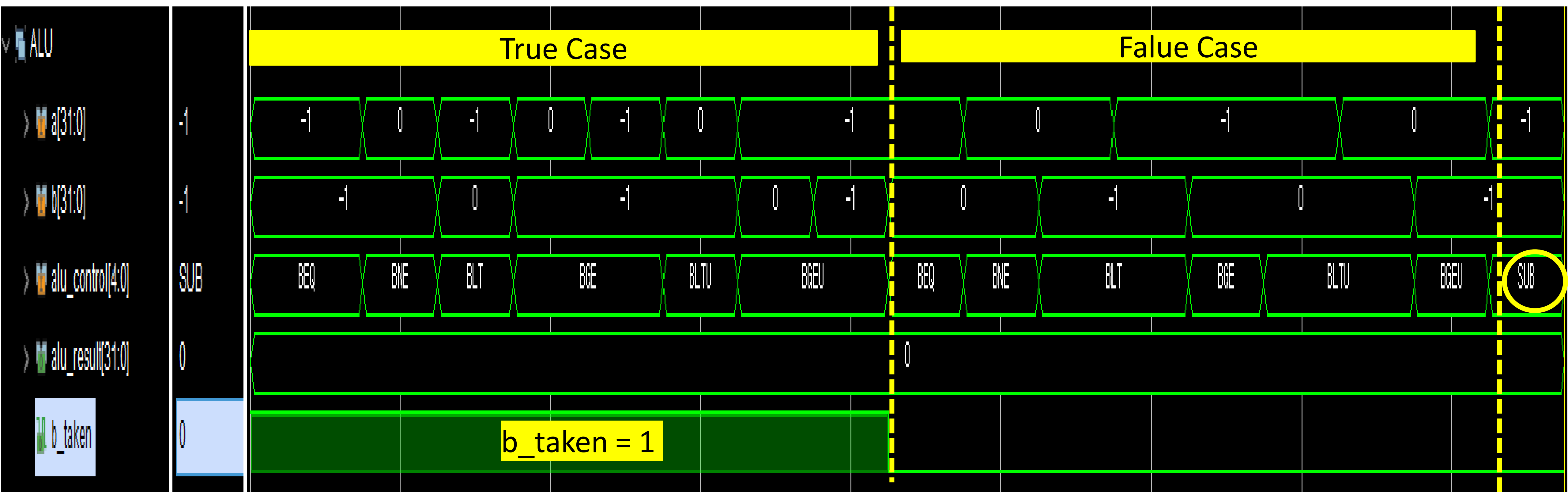
The main instruction diagram shows the following fields and values:

- 00
- 71
- 01
- 03
- 20
- 0
- 0
- 0
- SUB
- 0
- 1
- 0
- 0

control_unit	
OUTPUT	B_type
rf_we	0
alu_src_sel	0
rf_wd_sel	0
alu_control	1,0,func3
[b_taken,ALU_ctrl]	
b_src_sel	0
branch	1
o_func3	0
dwe	0

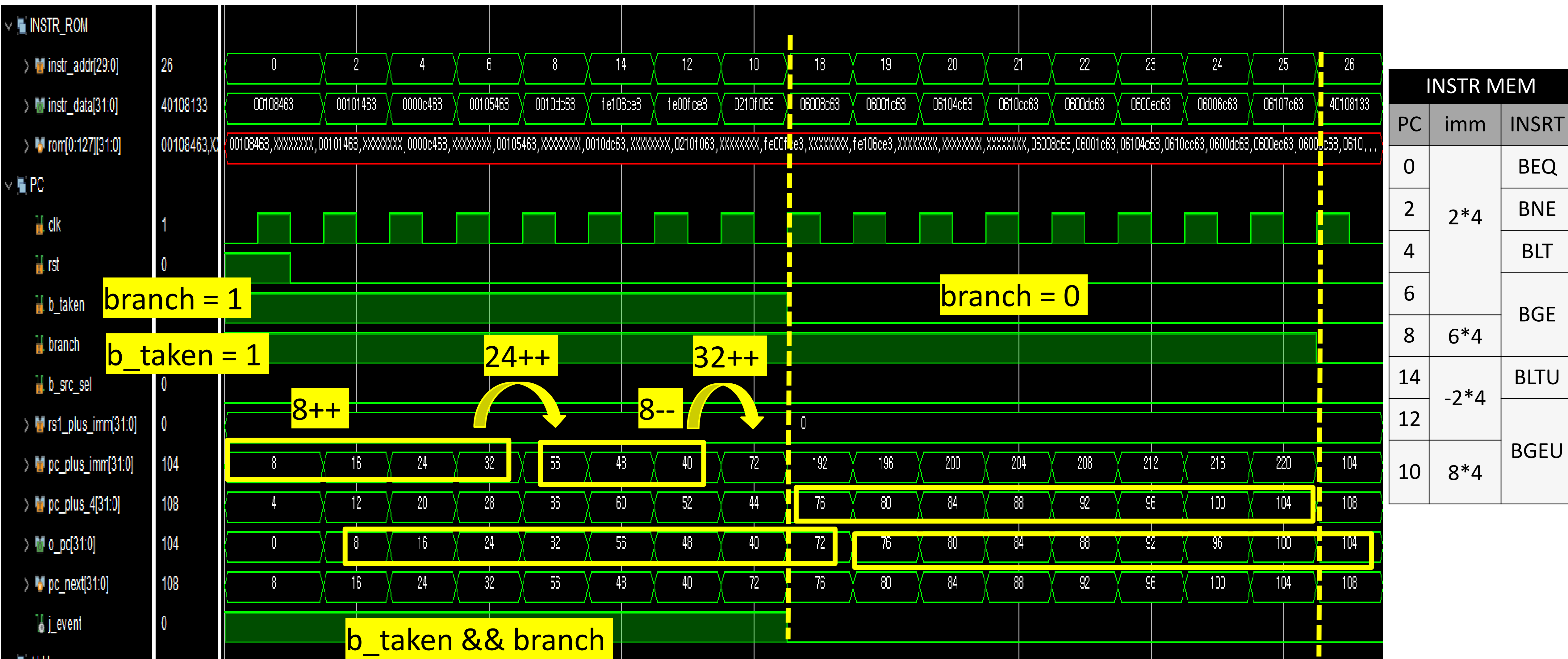
3. B-type

- simulation(ALU)



3. B-type

- simulation(PC)



3. U-type

OPCODE	INSTR	
7'b011_0111	LUI	Rd = imm
7'b001_0111	AUIPC	Rd = PC + imm

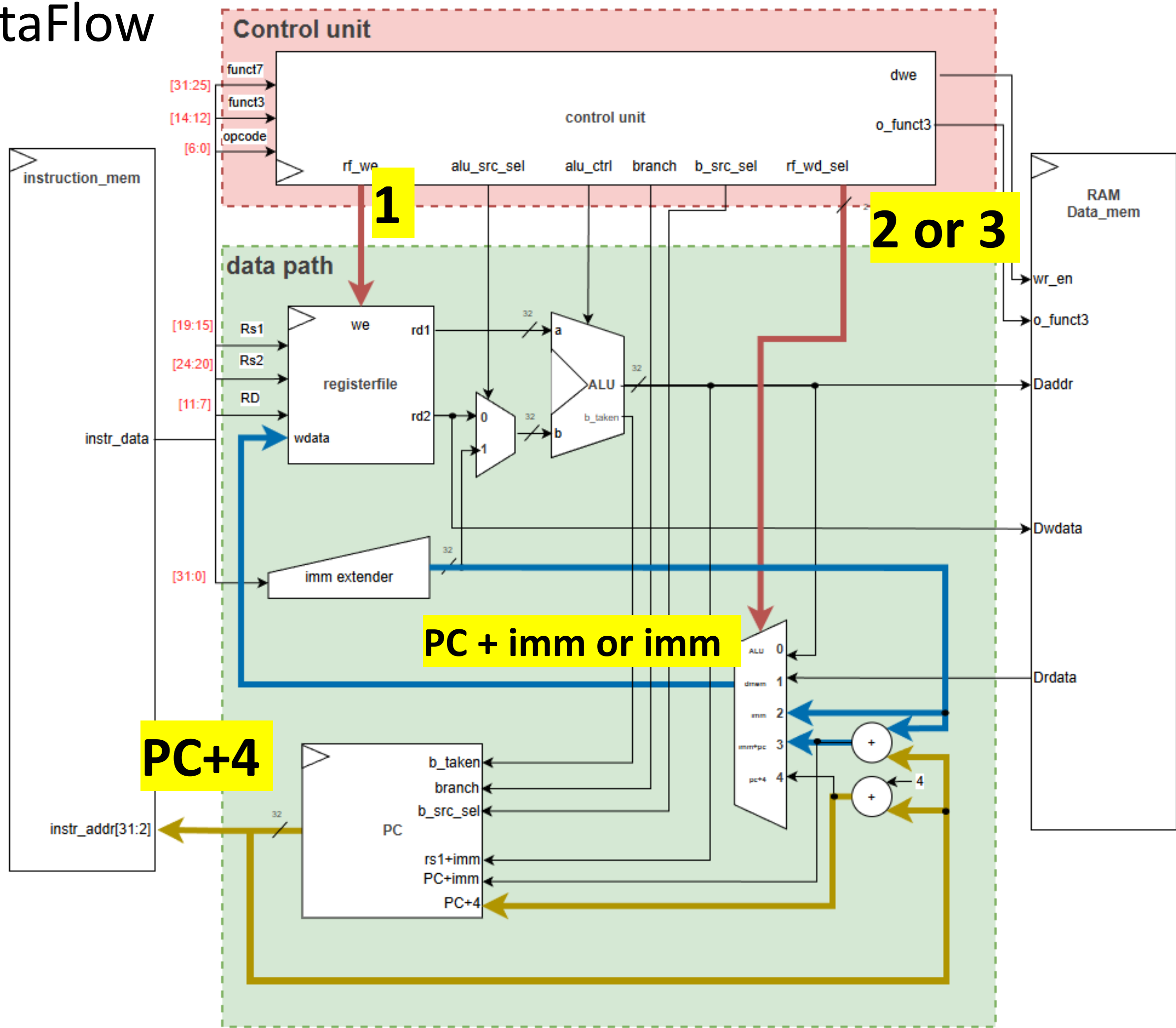
- OPCODE의 차이로 LUI,AUIPC 구별

U-type

Imm[31:12]	Rd	OPCODE(U-type)
------------	----	----------------

3. U-type

- DataFlow



control_unit		
OUTPUT	LUI_type	AUIPC_type
rf_we	1	
alu_src_sel	0	
rf_wd_sel	2	3
alu_control [b_taken,ALU_ctrl]	ADD	
b_src_sel	0	
branch	0	
o_funct3	0	
dwe	0	

U-type	Imm[31:12]	Rd	OPCODE(U-type)
--------	------------	----	----------------

3. U-type

- simulation(시나리오)

LUI : $rd = imm$
AUIPC : $rd = PC + imm$

register file	
addr	용도
x1	rd (LUI, AUIPC 저장소)



INSTR MEM			
PC	INSRT	설명	imm
0	BEQ	if(0 == 0)? PC += 2*4	
2	LUI	x1 = -10	-10
3	AUIPC	x1 = 3*4 + 10	10

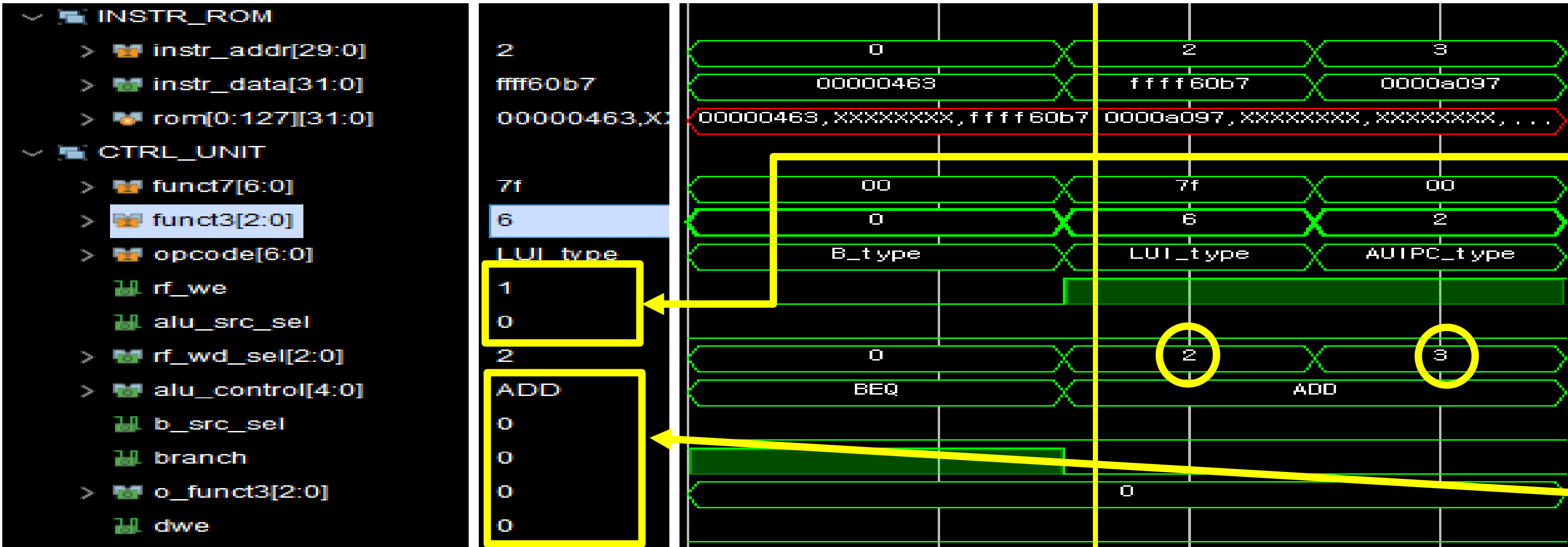
검증할 부분

추가한 이유

U-type는 0번 PC에서 simulation을 진행할 시,
imm과 imm+PC의 값이 동일하여 DATAFLOW 명확하게 확인 불가

3. U-type

- simulation(control Unit & RF Wdata MUX)

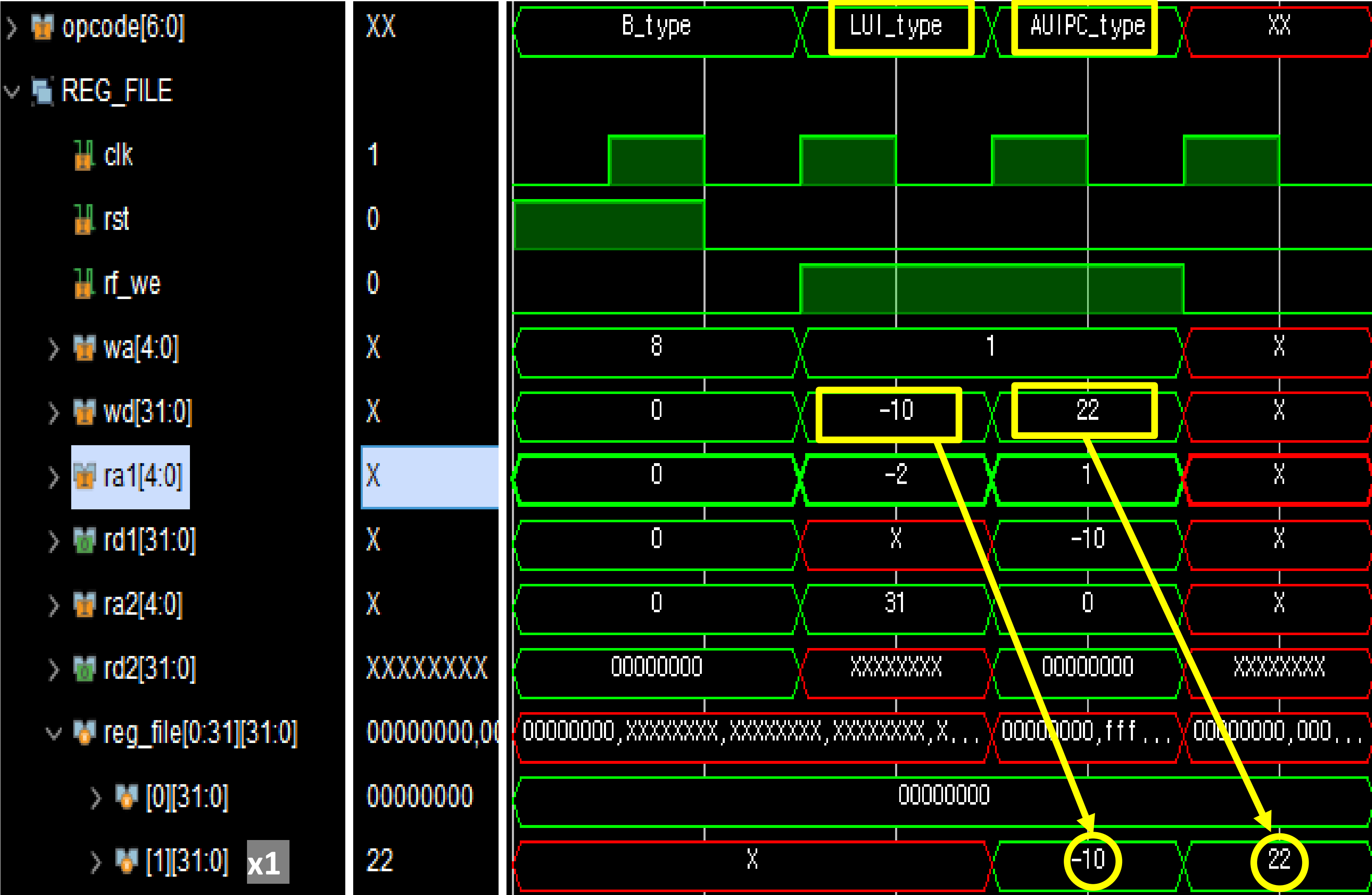


control_unit		
OUTPUT	LUI_type	AUIPC_type
rf_we	1	
alu_src_sel	0	
rf_wd_sel	2	3
alu_control [b_taken,ALU_ctrl]	ADD	
b_src_sel	0	
branch	0	
o_funct3	0	
dwe	0	

INSTR MEM			
PC	INSRT	register file	imm
2	LUI	x1 = -10	-10
3	AUIPC	x1 = 22	10

3. U-type

- simulation(Register File)



INSTR MEM			
PC	INSRT	register file	imm
2	LUI	x1 = -10	-10
3	AUIPC	x1 = 3*4 + 10	10

3. J-type

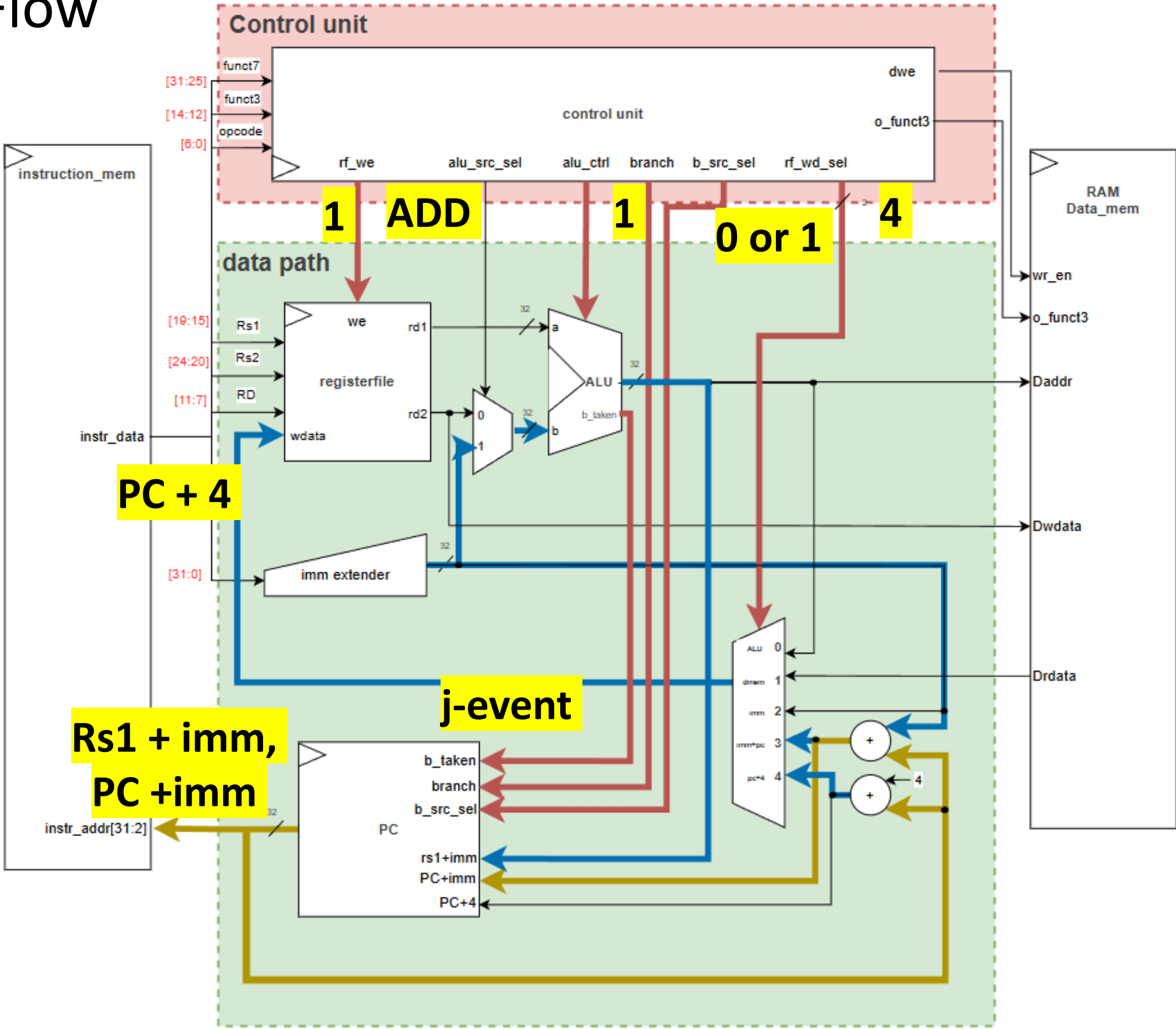
OPCODE	INSTR	Rd	PC
7'b110_1111	JAL	Rd = PC + 4	PC += imm
7'b110_0111	JALR		PC = Rs1 + imm

- J-type은 OPCODE로 구별
- PC는 무조건 Jump

JAL 명령어:	Imm[20,10:1,11,19:12]			Rd	OPCODE(JAL)
JALR명령어:	Imm[11:0]	rs1	funct3	Rd	OPCODE(JALR)

3. J-type

• DataFlow



control_unit		
OUTPUT	JAL_type	JALR_type
rf_we	1	
alu_src_sel	0	1
rf_wd_sel	4	
alu_control	JUMP	
[b_taken,ALU_ctrl]	B_taken:1	ALU:ADD
b_src_sel	0	1
branch	1	
o_funct3	0	
dwe	0	

PC		OUTPUT
Case		
j_event	b_src_sel	(pc_next)
0	x	PC + 4
1	0	PC + imm
	1	rs1 + imm

JAL 명령어:

Imm[20,10:1,11,19:12]

Rd

OPCODE(JAL)

JALR명령어:

Imm[11:0]

rs1

funct3

Rd

OPCODE(JALR)

3. J-type

- simulation(시나리오)

JAL : $rd = PC + 4$; $PC += imm$
JALR : $rd = PC + 4$; $PC = rs1 + imm$

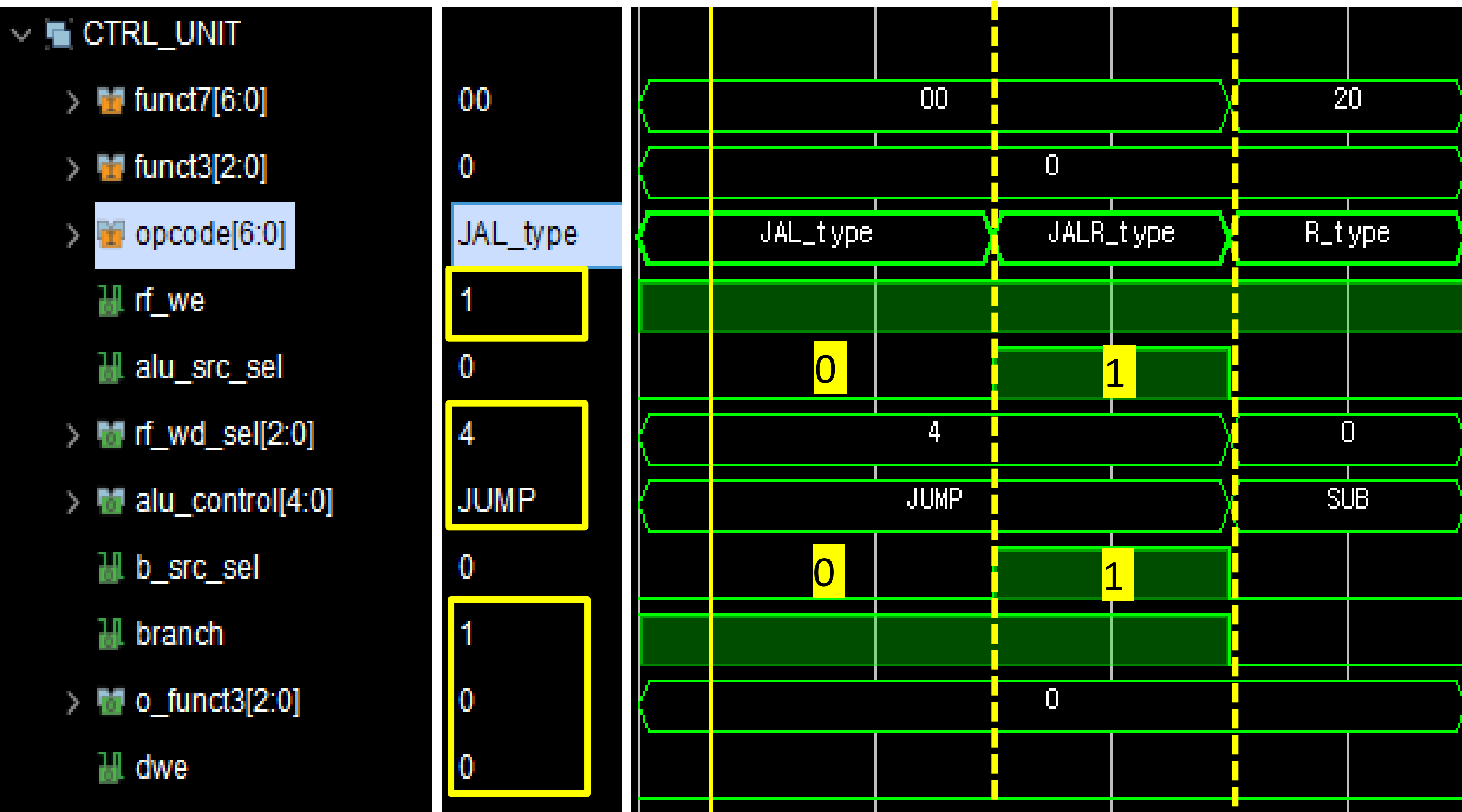
register file	
addr	용도
x1	JAL 저장소
x2	JALR 저장소
x3	SUB 값 저장소
x5	JALR rs1 값 $4*4$



INSTR MEM				
PC	INSRT	registerFile	PC	imm
0	JAR	$x1 = 0 + 4$	$PC = 0 + 3*4 = 3*4$	3*4
3	JALR	$x2 = 3*4 + 4$	$PC = x5(4*4) + 3*4 = 7*4$	
7	SUB	$x3 = x2 - x1 = 12$		

3. J-type

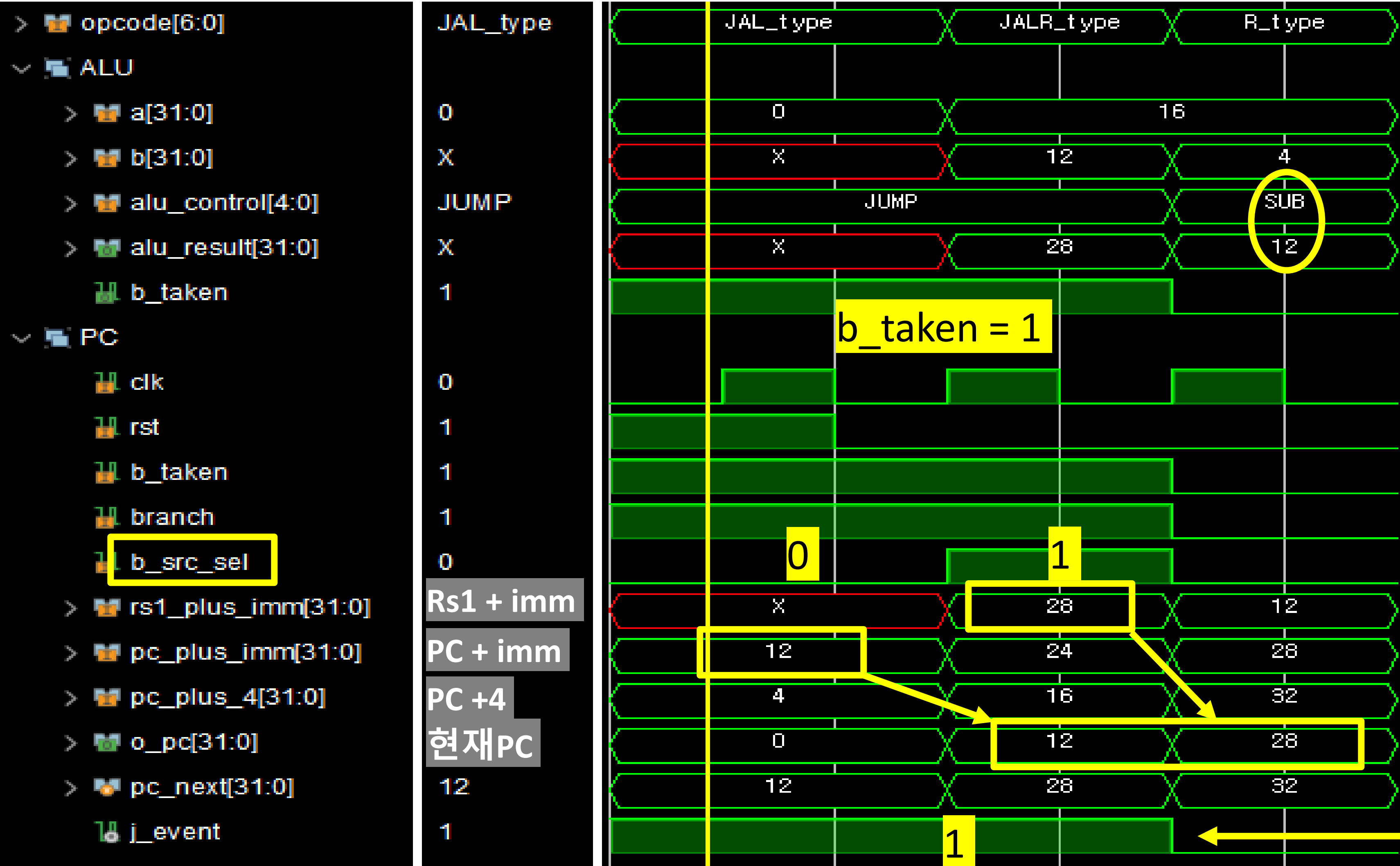
- simulation(Control Unit)



control_unit		
OUTPUT	JAL_type	JALR_type
rf_we	1	
alu_src_sel	0	1
rf_wd_sel	4	
alu_control	JUMP	
[b_taken,ALU_ctrl]	B_taken:1	ALU:ADD
b_src_sel	0	1
branch	1	
o_funct3	0	
dwe	0	

3. J-type

- simulation(ALU & PC)



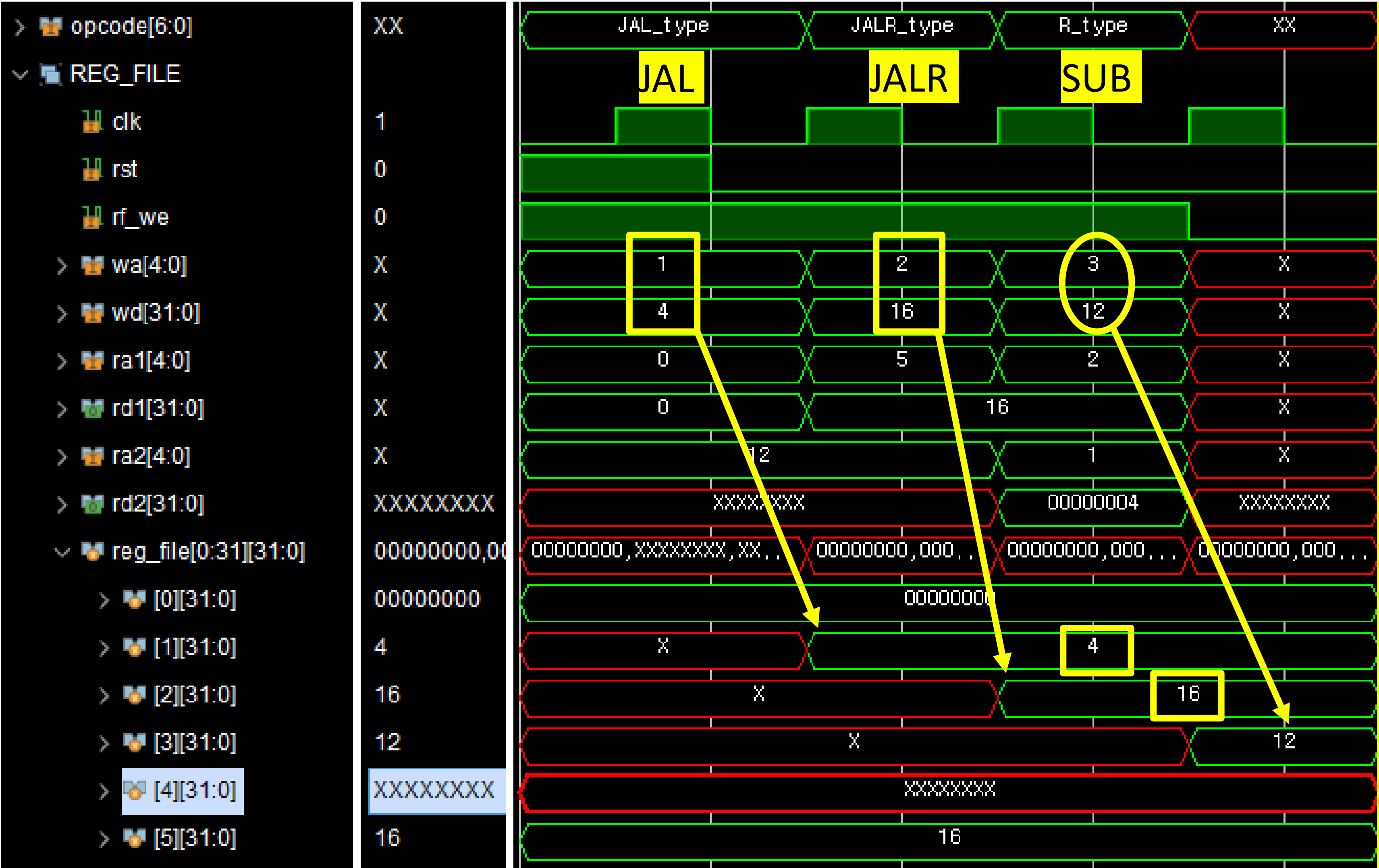
PC		
Case		OUTPUT
j_event	b_src_sel	(pc_next)
0	x	PC + 4
1	0	PC + imm
1	1	rs1 + imm

INSTR MEM			
PC	INSRT	registerFile	PC
0	JAR	x1 = 4	PC = 3*4
3	JALR	x2 = 16	PC = 7*4
7	SUB	x3 = 12	

j_event= branch && b_taken

3. J-type

- simulation(Register File)

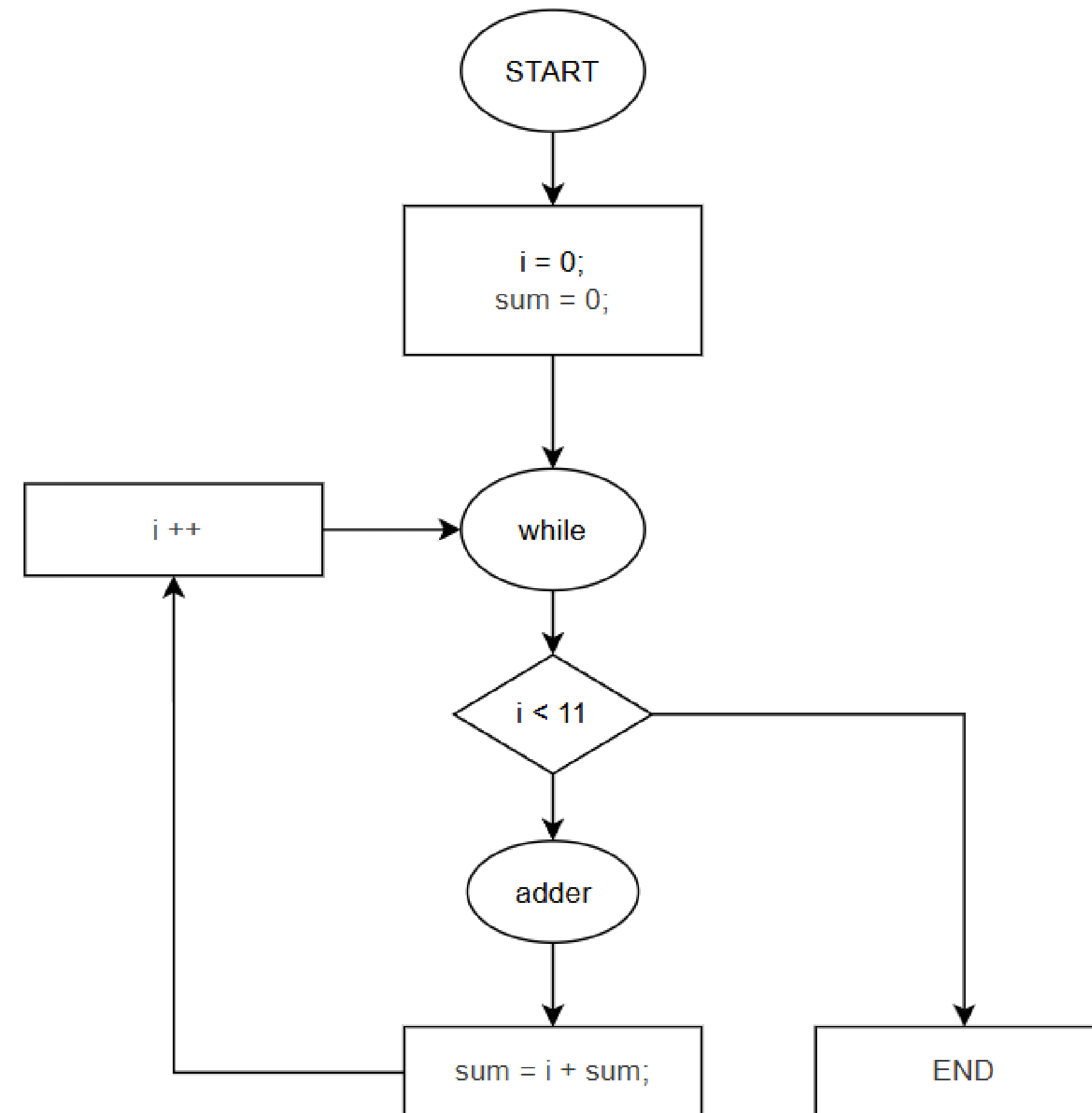


INSTR MEM			
PC	INSRT	registerFile	PC
0	JAR	x1 = 4	PC = 3*4
3	JALR	x2 = 16	PC = 7*4
7	SUB	x3 = 12	

4. C to Assembler

- 누적 sum program

```
1  int adder(int a, int b);
2
3
4  void main(void) {
5
6      int i = 0;
7
8      int sum = 0;
9
10     while(i<11){
11
12         sum = adder(i,sum);
13
14         i = i + 1;
15     }
16
17
18 }
19
20 int adder(int a, int b){
21
22     return a + b;
23
24 }
```



4. C to Assembler

- 누적 sum program

start → 전처리 → 함수 → 후처리 → while → END

start

함수 호출 전처리

함수 return 후처리

While문

End

```
0000000000000000 :
0: 19000113      li      sp,400

0000000000000004 :
4: fe010113      addi     sp,sp,-32
8: 00112e23      sw       ra,28(sp)
c: 00812c23      sw       s0,24(sp)
10: 02010413      addi     s0,sp,32
14: fe042623      sw       zero,-20(s0)
18: fe042423      sw       zero,-24(s0)
1c: 0200006f      j        3c
20: fe842583      lw       a1,-24(s0)
24: fec42503      lw       a0,-20(s0)
28: 038000ef      jal      ra,60
2c: fea42423      sw       a0,-24(s0)
30: fec42783      lw       a5,-20(s0)
34: 00178793      addi     a5,a5,1
38: fef42623      sw       a5,-20(s0)
3c: fec42703      lw       a4,-20(s0)
40: 00a00793      li       a5,10
44: fce7dee3      ble     a4,a5,20
48: 00000013      nop
4c: 00000013      nop
50: 01c12083      lw       ra,28(sp)
54: 01812403      lw       s0,24(sp)
58: 02010113      addi     sp,sp,32
5c: 00008067      ret
```

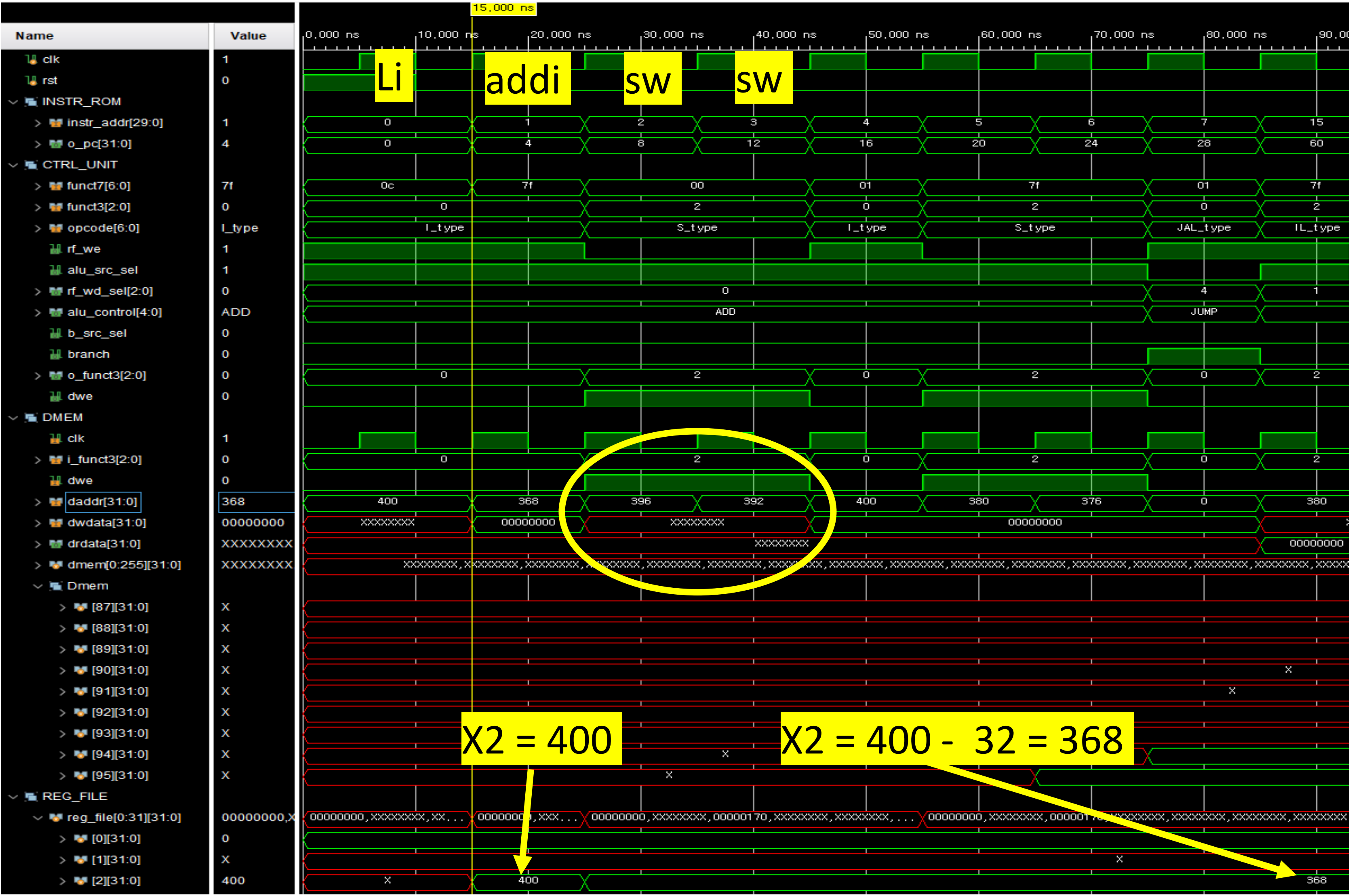
0000000000000060 :

60: fe010113 addi sp,sp,-32
64: 00112e23 sw ra,28(sp)
68: 00812c23 sw s0,24(sp)
6c: 02010413 addi s0,sp,32
70: fea42623 sw a0,-20(s0)
74: feb42423 sw a1,-24(s0)
78: fec42703 lw a4,-20(s0)
7c: fe842783 lw a5,-24(s0)
80: 00f707b3 add a5,a4,a5
84: 00078513 mv a0,a5
88: 01c12083 lw ra,28(sp)
8c: 01812403 lw s0,24(sp)
90: 02010113 addi sp,sp,32
94: 00008067 ret

Adder 함수

4. C to Assembler

- 누적 sum program



```
0000000000000000:
0: 19000113 li sp,400
0000000000000004:
:
4: fe010113 addi sp,sp,-32
8: 00112e23 sw ra,28(sp)
c: 00812c23 sw s0,24(sp)
```

Annotations:

- 스택 포인터 Sp을 400주소로 지정
- Sp을 -32까지 확보
- 함수가 끝난 뒤 return 주소 저장
- 현 함수를 호출하기 전 단계의 기준 s0주소 저장

8,C: 함수를 호출하지 않았기 때문에 SW 동작 의미 X

register file			Data MEM	
addr	목적	value	addr	value
x2	SP	400 -> 368		

4. C to Assembler



```
10: 02010413 addi s0,sp,32
14: fe042623 sw zero,-20(s0)
18: fe042423 sw zero,-24(s0)
1c: 0200006f j 3c
```

S0을 400주소로 저장

I = 0;

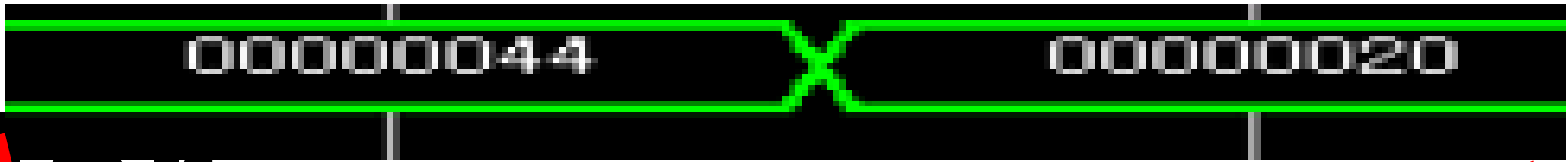
Sum = 0;

3C(while문)로 JUMP

register file		
addr	목적	value
x8	S0	400
x2	SP	368

Data MEM		
addr	목적	value
95	I	0
94	Sum	0

4. C to Assembler



Load I 값

비교값
10 저장

If(i <= 10), JUMP 20

```
3c: fec42703    lw    a4,-20(s0)
40: 00a00793    li    a5,10
44: fce7dee3    ble   a4,a5,20
```

Name

Value

clk

1

INSTR_ROM

instr_addr[29:0]

7

o_pc[31:0]

0000001c

CTRL_UNIT

opcode[6:0]

JAL_type

alu_control[4:0]

JUMP

DMEM

clk

1

i_funct3[2:0]

0

dwe

0

daddr[31:0]

0

Dmem

[87][31:0]

X

[88][31:0]

X

[89][31:0]

X

[90][31:0]

X

[91][31:0]

X

[92][31:0]

X

[93][31:0]

X

[94][31:0]

0

[95][31:0]

0

REG_FILE

reg_file[0:31][31:0]

00000000,X

[0][31:0]

0

[1][31:0]

X

[2][31:0]

368

[3][31:0]

X

[4][31:0]

X

[5][31:0]

X

[6][31:0]

X

[7][31:0]

X

[8][31:0]

400

[9][31:0]

X

[10][31:0]

X

[11][31:0]

X

[12][31:0]

X

[13][31:0]

X

[14][31:0]

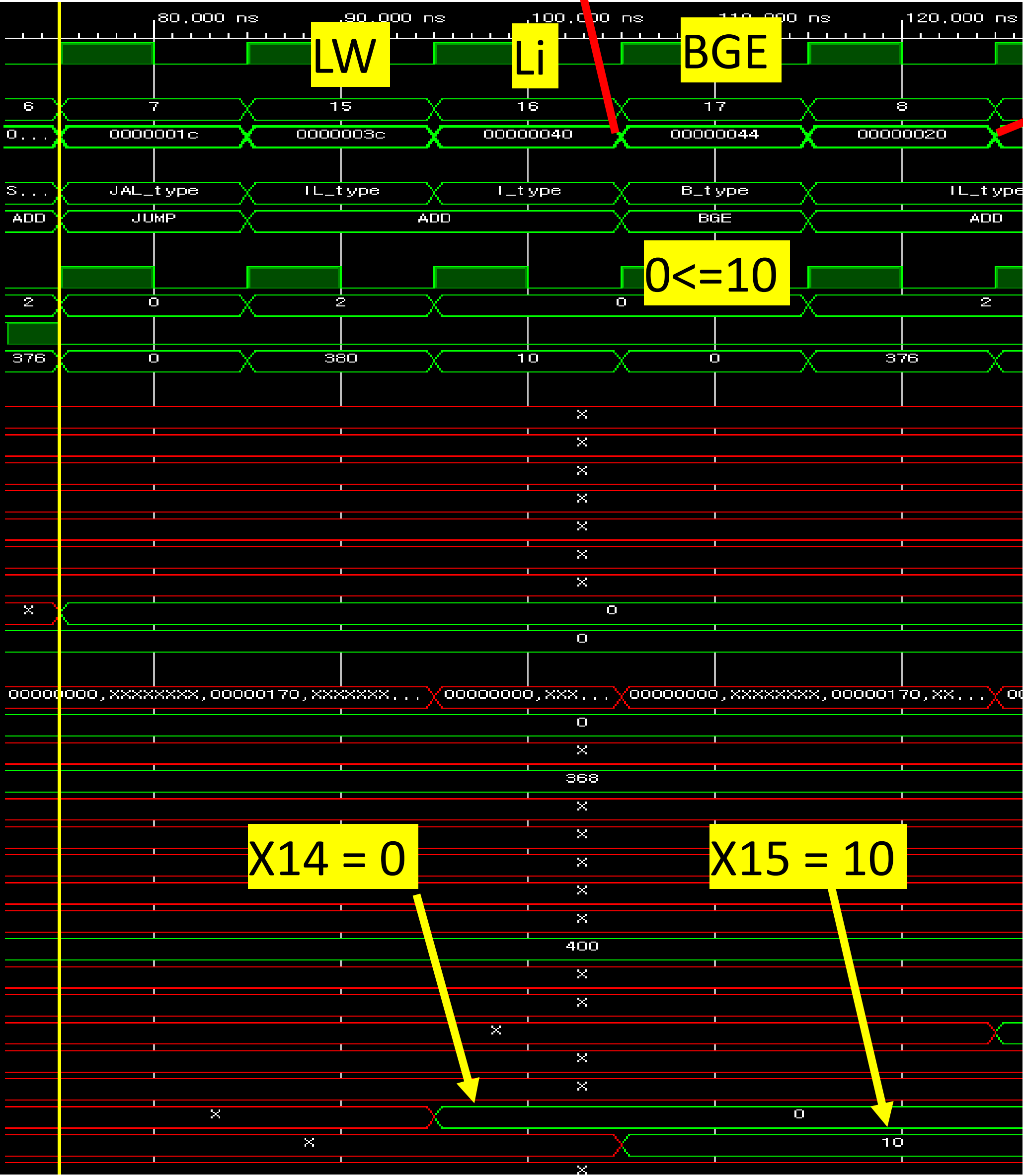
X

[15][31:0]

X

[16][31:0]

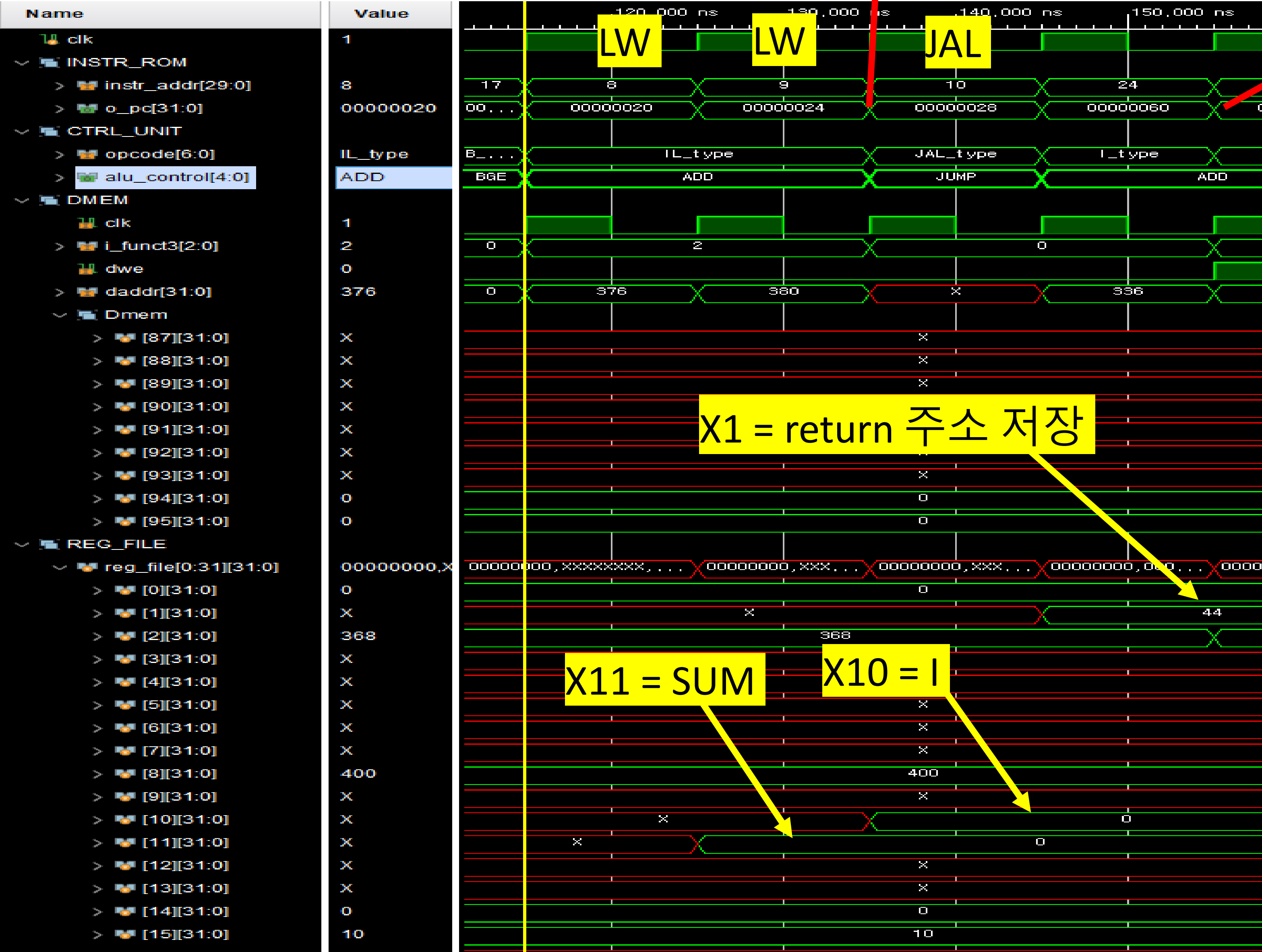
X



register file		
addr	목적	value
x15	비교값	10
x14	I	0
x8	S0	400
x2	SP	368

Data MEM		
addr	목적	value
95	I	0
94	Sum	0

4. C to Assembler



20: fe842583 lw a1,-24(s0)
24: fec42503 lw a0,-20(s0)
28: 038000ef jal ra,60

Load SUM

Load I

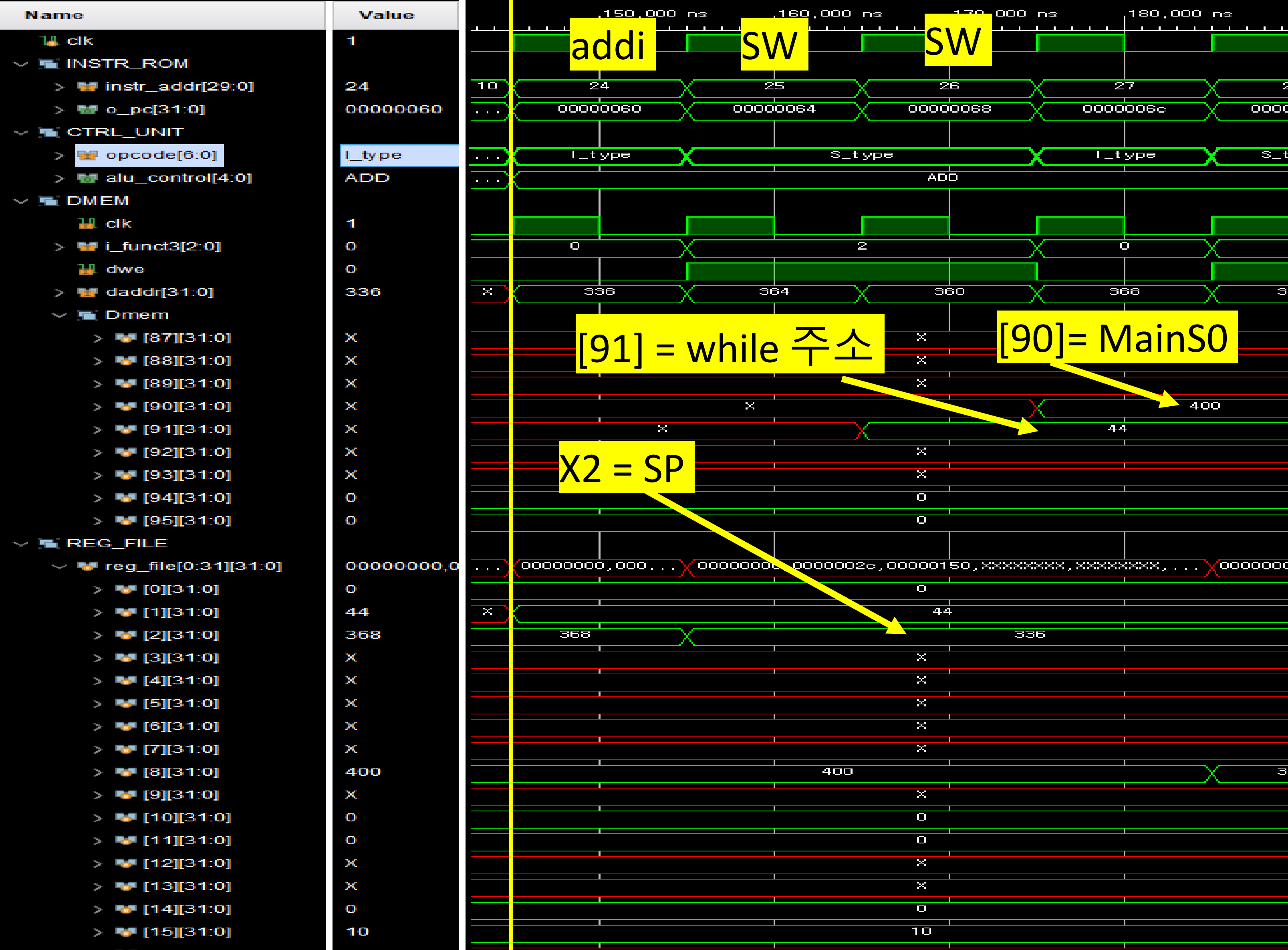
JUMP 60
Adder 함수 호출 및
Retrun 주소 저장
44 = while 조건문

44: fce7dee3 ble a4,a5,20

register file		
addr	목적	value
x15	비교값	10
x14	I	0
X11	SUM	0
X10	I	I
x8	S0	400
x2	SP	368
X1	While문 주소	44

Data MEM		
addr	목적	value
95	I	0
94	Sum	0

4. C to Assembler



```
000000000000000060 :
60: fe010113      addi sp,sp,-32
64: 00112e23      sw  ra,28(sp)
68: 00812c23      sw  s0,24(sp)
```

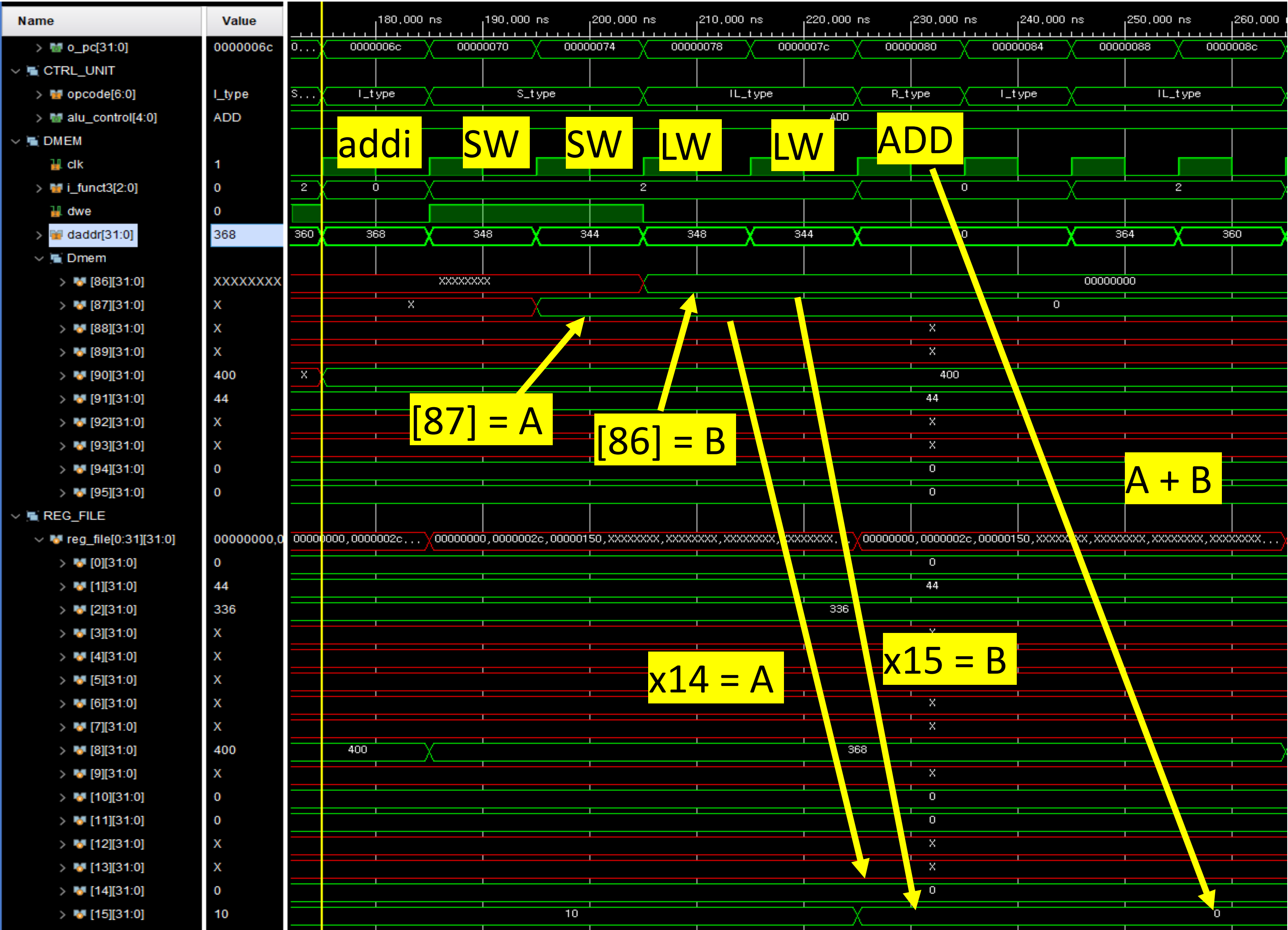
mine S0 값 저장

함수 끝나면
돌아갈 주소 저장

register file		
addr	목적	value
x15	비교값	10
x14	I	0
X11	SUM	0
X10	I	I
x8	S0	400
x2	SP	336
X1	While문 주소	44

Data MEM		
addr	목적	value
95	I	0
94	Sum	0
91	Return 주소	44
90	Main S0	400

4. C to Assembler

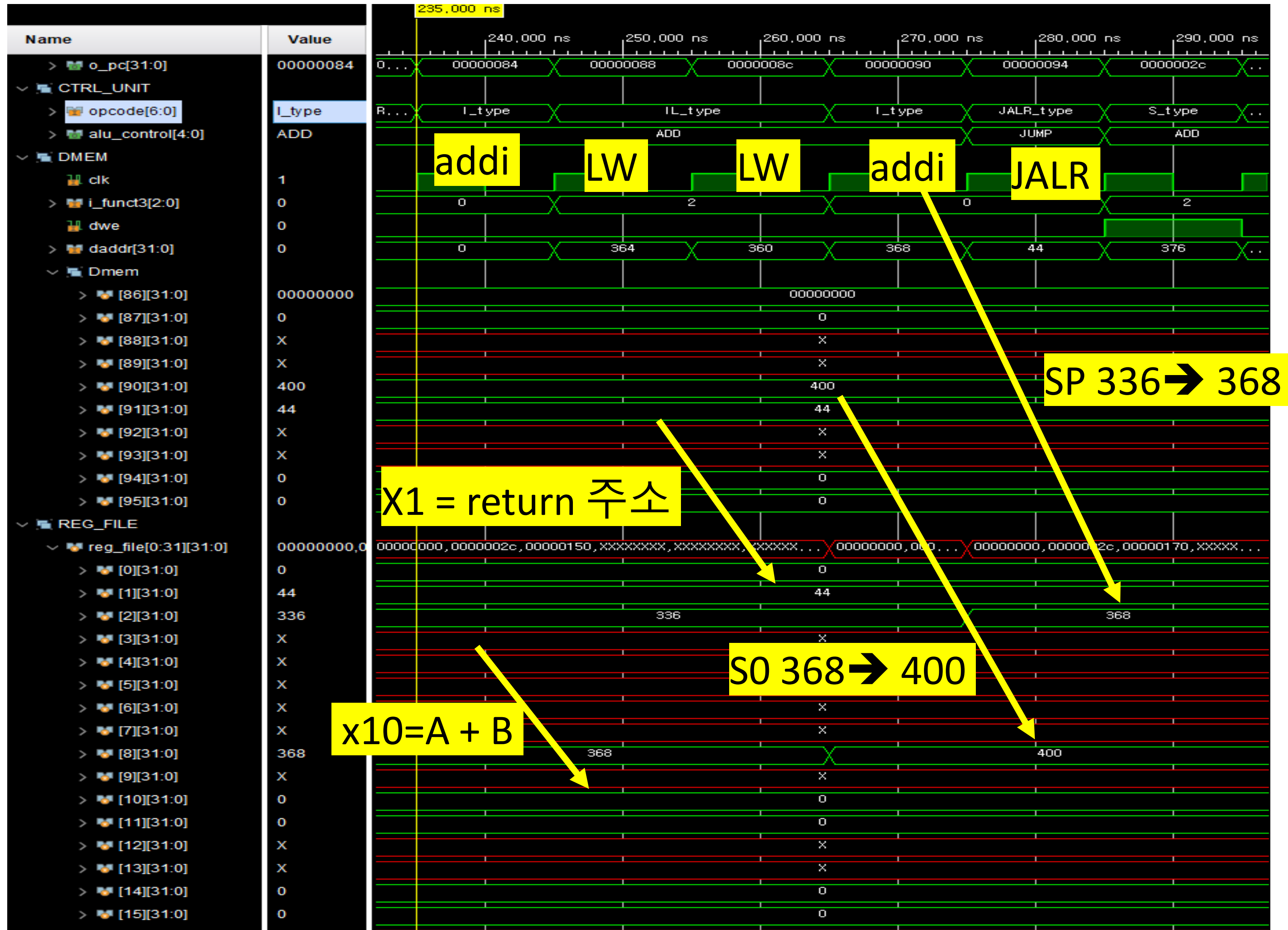


```
6c: 02010413      addi s0,sp,32    s0 이동
70: fea42623      sw  a0,-20(s0)   A 인자 저장
74: feb42423      sw  a1,-24(s0)   B 인자 저장
78: fec42703      lw  a4,-20(s0)   A 인자 Load
7c: fe842783      lw  a5,-24(s0)   B 인자 Load
80: 00f707b3      add  a5,a4,a5     A + B
```

register file		
addr	목적	value
x15	A+B	0
x14	A인자	0
X11	SUM	0
X10	I	I
x8	S0	368
x2	SP	336
X1	While문 주소	44

Data MEM		
addr	목적	value
95	I	0
94	Sum	0
91	Return 주소	44
90	Main S0	400
87	A인자	0
86	B인자	0

4. C to *Assembler*

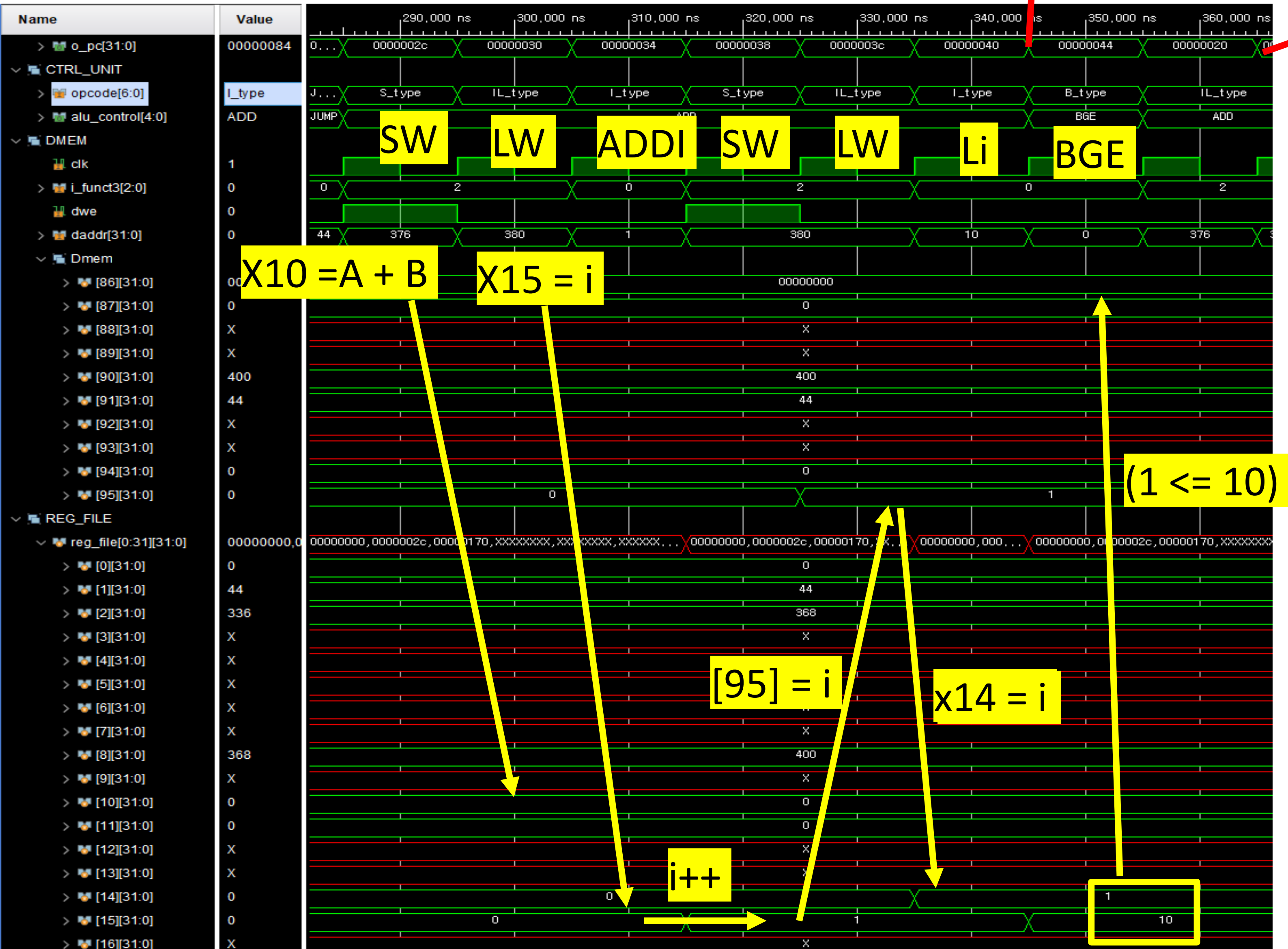
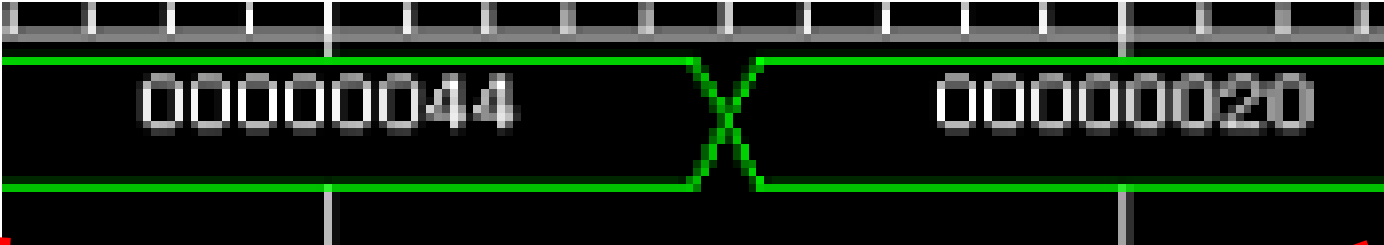


84: 00078513	mv a0,a5	A + B을 x10으로 이동
88: 01c12083	lw ra,28(sp)	Return 주소 Load
8c: 01812403	lw s0,24(sp)	함수 s0,SP
90: 02010113	addi sp,sp,32	반납
94: 00008067	ret	Return (adder 함수 끝)

register file		
addr	목적	value
x15	A+B	0
x14	A인자	0
X11	SUM	0
X10	A+B	0
x8	S0	400
x2	SP	368
X1	While문 주소	44

Data MEM		
addr	목적	value
95	I	0
94	Sum	0
91	Return 주소	44
90	Main S0	400
87	A인자	0
86	B인자	0

4. C to Assembler

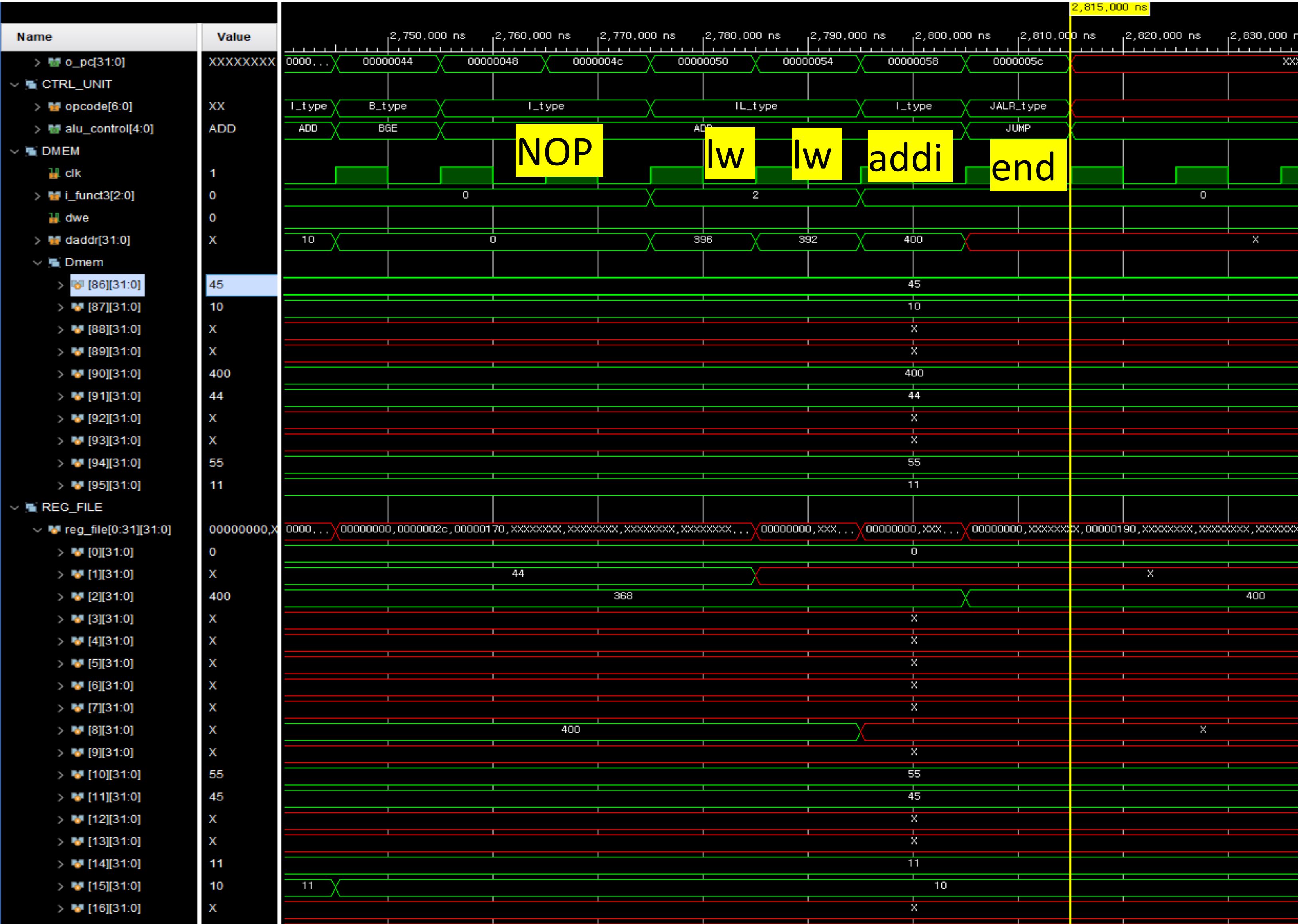


2c:	fea42423	sw	a0,-24(s0)	A +B 저장
30:	fec42783	lw	a5,-20(s0)	i을 Load
34:	00178793	addi	a5,a5,1	i++
38:	fef42623	sw	a5,-20(s0)	i을 저장
3c:	fec42703	lw	a4,-20(s0)	i을 Load
40:	00a00793	li	a5,10	i을 Load
44:	fce7dee3	ble	a4,a5,20	If(i<= 10), 20이동

register file		
addr	목적	value
x15	i	1
x14	i	1
X11	SUM	0
X10	A+B	0
x8	S0	400
x2	SP	368
X1	While문 주소	44

Data MEM		
addr	목적	value
95	I	0
94	Sum	0
91	Return 주소	44
90	Main S0	400
87	A인자	0
86	B인자	0

4. C to Assembler



48: 00000013 nop
4c: 00000013 nop
50: 01c12083 lw ra,28(sp)
54: 01812403 lw s0,24(sp)
58: 02010113 addi sp,sp,32
5c: 00008067 ret

SKIP

복귀할 주소 Load
Main : 없음

s0 반납

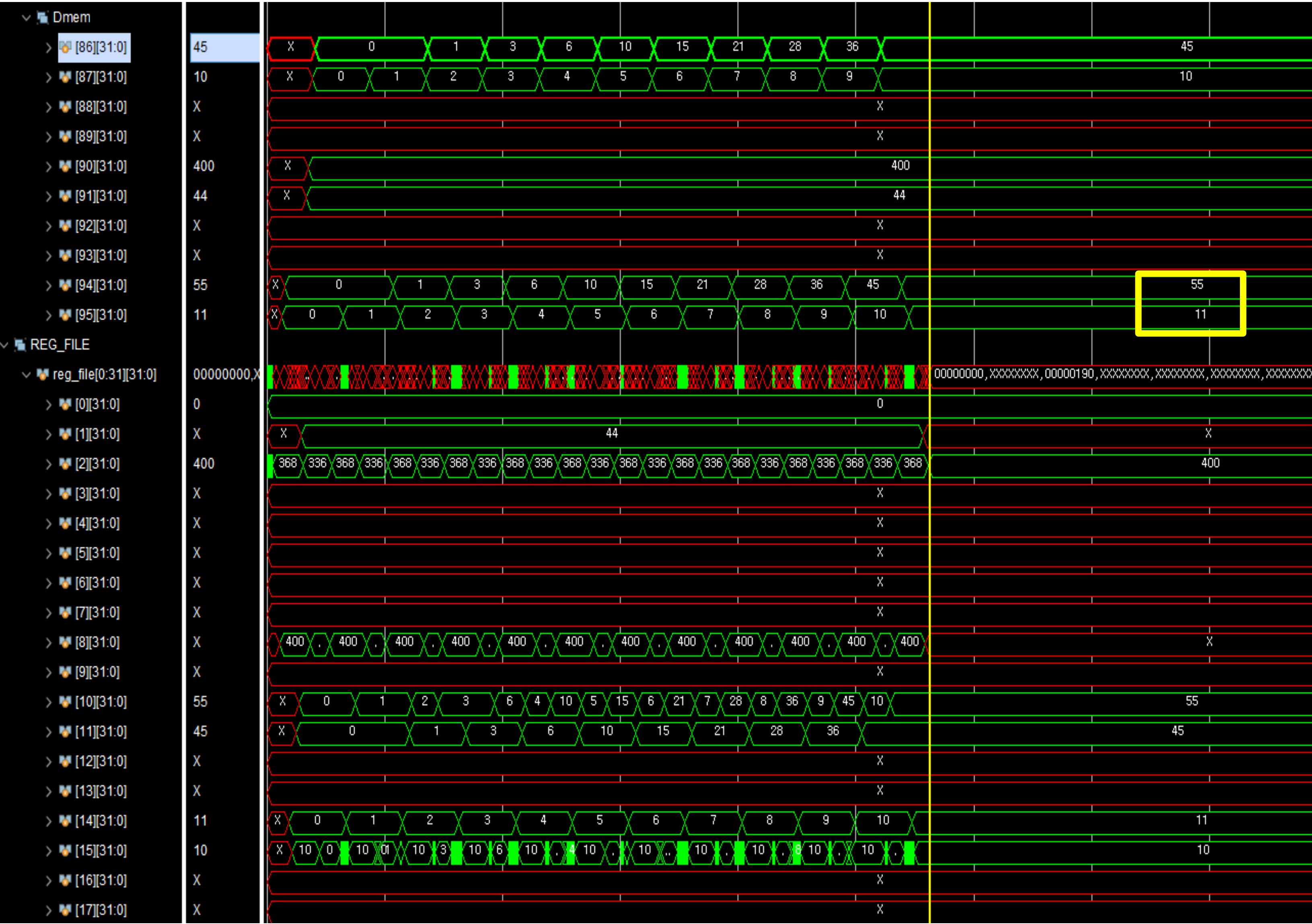
Sp 반납

end

register file		
addr	목적	value
x15	i	10
x14	i	11
X11	SUM	45
X10	A+B	55
x8	S0	xx
x2	SP	Xx
X1	While문 주소	xx

Data MEM		
addr	목적	value
95	I	11
94	Sum	55
91	Return 주소	44
90	Main S0	400
87	A인자	10
86	B인자	45

4. C to Assembler



SUM = 55
I = 11

register file		
addr	목적	value
x15	i	10
x14	i	11
X11	SUM	45
X10	A+B	55
x8	S0	xx
x2	SP	Xx
X1	While문 주소	xx

Data MEM		
addr	목적	value
95	I	11
94	Sum	55
91	Return 주소	44
90	Main S0	400
87	A인자	10
86	B인자	45

5. Trouble shooting

- **힘든 점:**

누적 sum program을 분석하는 과정에서 전체적인 Assembler Code구조에 이해가 힘들음
느낌

(Register와 data memory을 쓰임새, 함수 호출 시 새로운 스택을 사용)

- **해결방안:**

Data Flow을 분석을 여러 차례 분석을 진행한 결과 규칙적인 쓰임새가 보이기 시작함
규칙성을 바탕으로 위의 문제점을 해결함